Part III

Mathematical Foundations of SLAM

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part III The Mathematical Foundations of SLAM

Chapter 10

Coordinate Systems and Rigid-Body Motion

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part II has introduced all the sensors, but each of them speaks for itself—the LiDAR says "there is a wall three meters ahead," the camera says "there is a corner at image coordinates (320, 240)," and the IMU says "it turned 0.3 degrees about its own X axis." These statements do not even share units or a reference, and they cannot be compared at all.

What Part III does is make them speak one language. This chapter is the first and most basic step: establish coordinate systems and learn to move any piece of data from one frame into another.

It is the most mathematically dense chapter in the book, and also the one that repays the most. From the next chapter onward, the symbols of this chapter appear in every expression: the state x of Chapter 11 is the pose of this chapter, the variables optimized in Chapter 12 are a string of poses, placing LiDAR points into a map in Chapter 13 uses the 2D transformation of this chapter, the ICP of Chapter 17 solves for a T, and ORB-SLAM2 in Chapter 22 tracks a T for every frame. One might say that every chapter after this one is solving for some kind of T.

Figure 10-0 How Chapter 10 Connects to the Rest of the Book

[Building on the previous chapters]

Chapters 4 through 9 Six classes of sensor, each speaking in its own frame and its own units

This chapter Establishing a common language: frames, pose, transformation matrices

[This chapter is responsible for]

The three levels of frame: world → robot → sensor

Homogeneous coordinates: writing "rotation + translation" as a single matrix multiplication

Three representations of rotation and the trade-offs among them (Table 10-1)

The TF tree: how ROS manages the whole tree of frames

[Where it leads]

→ Chapter 11 The pose becomes the "state x" to be estimated

→ Chapter 12 The nodes of a pose graph are the T of this chapter

→ Chapter 13 Putting LiDAR points into a map: a direct application of the formulation of Section 10.3

→ Chapter 17 ICP registration: solving for a T between two point clouds

→ Chapters 28 and 29 A faulty TF tree is the most common failure in ROS mapping

Learning Objectives

Part II introduced all of the robot's senses, but every sensor reports its data in its own frame—the camera says "there is something two meters ahead," the LiDAR says "there is a wall three meters to my left." To assemble this information into one consistent map, one must first learn to convert precisely among different frames. This chapter is the first stop of Part III and the bedrock of all the mathematics of SLAM. After completing this chapter, you should be able to:

1. Describe the relationship among the world, robot, and sensor frames, and explain why SLAM needs several frames.
2. Use homogeneous coordinates to express rotation and translation together as a single matrix operation.
3. Write the representation of a 2D pose (x, y, θ) and the coordinate transformation formulas.
4. Describe the three representations of 3D rotation—rotation matrix, Euler angles, and quaternion—with their merits, drawbacks, and proper settings.
5. Describe the composition and inversion of coordinate transformations, and understand the concept of the TF tree.
6. Describe the meaning of and the relations among the standard ROS frames map, odom, base_link, and camera_link.

Core Concepts

Chinese Term	English Term	Explanation
座標系	Coordinate Frame	The reference against which position and direction are described; SLAM has several at once.
位姿	Pose	Position plus attitude; (x, y, θ) in 2D, position plus three axes of rotation in 3D.
剛體運動	Rigid Body Motion	Motion that does not change an object's shape or size, consisting only of rotation and translation.
齊次座標	Homogeneous Coordinates	A coordinate representation with one extra dimension, letting rotation and translation be expressed by a single

Chinese Term	English Term	Explanation
		matrix.
旋轉矩陣	Rotation Matrix	An orthogonal matrix describing rotation, belonging to the special orthogonal group SO(3).
歐拉角	Euler Angles	Rotation expressed by the three angles roll, pitch, and yaw; intuitive but subject to gimbal lock.
四元數	Quaternion	Rotation expressed by four numbers; free of singularities and computationally efficient, the standard representation in SLAM.
變換矩陣	Transformation Matrix	A 4×4 matrix containing both the rotation R and the translation t, belonging to SE(3).
萬向鎖	Gimbal Lock	The singular phenomenon in which Euler angles lose a degree of freedom at particular attitudes.
座標轉換樹	TF Tree	The tree structure that manages the relations among frames in ROS.
里程計座標	odom frame	A reference frame that is continuous and smooth but drifts over the long run.
地圖座標	map frame	A global reference frame that does not drift but may jump.

10.1 World, Robot, and Sensor Frames

Picture a robot in a classroom with a camera mounted on its head, which sees "a chair two meters straight ahead." A question hides in that statement: straight ahead of what? Of the camera, of the robot's body, or of the classroom? The three answers differ—turn the robot and "ahead" relative to the camera changes, while the chair's position relative to the classroom does not. This is why SLAM must handle several frames at once.

Three Levels of Frame

- World frame: a fixed global reference, usually with its origin at the robot's starting point at power-on or at some corner of the map. The absolute position of everything on the map is referred to it. It is the frame into which SLAM ultimately wants to unify everything.
- Robot (body) frame: a frame fixed to the robot and moving with it, with its origin usually at the center of the chassis. In ROS it is called base_link. The robot's motion is, in essence, the motion of this frame relative to the world frame.
- Sensor frame: fixed to each sensor. The camera has a camera frame and the LiDAR a LiDAR frame, with the origin at each sensor's measurement center. What a sensor measures directly is expressed in its own frame.

Why So Many Levels Are Needed

The key is this: where a sensor is mounted on the robot is fixed (the camera is always 20 centimeters forward of the head, the LiDAR always 15 centimeters above the chassis), while the robot's position relative to the world changes constantly. Separating the fixed mounting relationship from the changing robot pose is what keeps the

mathematics clear.

So a piece of sensor data entering the map takes two steps: first from the sensor frame into the robot frame (using the fixed mounting relationship, called the extrinsics, mentioned in Chapter 5), then from the robot frame into the world frame (using the robot's current pose, which is exactly what SLAM estimates). Understanding this sensor → robot → world chain is a prerequisite for reading any SLAM system. The rest of this chapter assembles the mathematical tools for doing the transforming.

10.2 Homogeneous Coordinates

A coordinate transformation involves two actions: rotation (turning a direction) and translation (moving a position). The nuisance is that rotation is multiplication (a matrix times a vector) while translation is addition (a vector plus a vector), and the two differ in form, which makes chaining several transformations verbose. Homogeneous coordinates are an elegant trick that unifies the two into a single matrix multiplication.

The Magic of One Extra Dimension

The method is simple: append a 1 to the original coordinates. A 2D point (x, y) is written (x, y, 1) and a 3D point (x, y, z) is written (x, y, z, 1).

The benefit is that translation can now be written into the matrix as well. In 2D, "rotate by θ then translate by (tx, ty)" can be merged into a single 3×3 matrix multiplication:

$$[x';y';1] = [\cos\theta \ -\sin\theta \ tx;\sin\theta \ \cos\theta \ ty;0 \ 0 \ 1]\cdot[x;y;1]$$

This 3×3 matrix is called the 2D transformation matrix (or homogeneous transformation matrix). Its upper-left 2×2 block is the rotation, its upper-right column the translation, and a row (0 0 1) is appended at the bottom. What once took two

steps—multiply by the rotation, then add the translation—is now done in one matrix multiplication.

Why This Matters

- Chaining transformations becomes a product: whether the robot has moved through several segments or the data passes through several layers of sensor → robot → world, one need only multiply the transformation matrices in order, with no separate handling of rotation and translation.
- The inverse transformation is the inverse matrix: to convert world coordinates back into robot coordinates, take the inverse of the transformation matrix.
- Uniform implementation: all coordinate handling is matrix multiplication, which keeps the code concise and less error-prone and lends itself to GPU acceleration.

The 3D case is entirely analogous, with a 4×4 matrix instead: the upper-left 3×3 block is the rotation matrix R, the upper-right 3×1 column the translation vector t, and a row (0 0 0 1) is appended at the bottom. This 4×4 matrix is one of the most central mathematical objects in SLAM: it carries the complete pose information of where the robot faces and where it is.

A Clarification: Two Ways of Reading a Transformation Matrix

The same transformation matrix T admits two readings, which beginners often confuse:

① Coordinate transformation: converting the coordinates of the same point in frame A into its coordinates in frame B.

② Rigid-body motion: actually moving and rotating a point to a new position (within the same frame).

The two have the same mathematical form but opposite meanings (turning a frame one way is the same as turning a point the other way).

In SLAM it is mostly the first—we do not really move the world, we describe the same thing in another frame.

10.3 The 2D Pose: x, y, θ

Cleaning robots and teaching vehicles move in a plane, and a 2D pose suffices to describe them. This is the best starting point for understanding 3D, because 2D has only one rotational degree of freedom and everything is intuitive.

Three Numbers Describe Everything

The pose of a robot in a plane is completely described by three numbers:

- x, y: the robot's position in the world frame.
- θ: the robot's heading angle, the angle between the robot's forward direction and the world frame's X axis.

This is exactly the quantity estimated by the dead reckoning of Chapter 3 and the Gmapping particles of Chapter 14. In 2D SLAM, every particle and every pose node is just such a triple (x, y, θ).

A Worked Example of 2D Coordinate Transformation

Suppose the LiDAR measures a point p_robot = (a, b) in the robot frame (a ahead of the robot, b to its left), and the robot's pose in the world frame is (x, y, θ). Then the

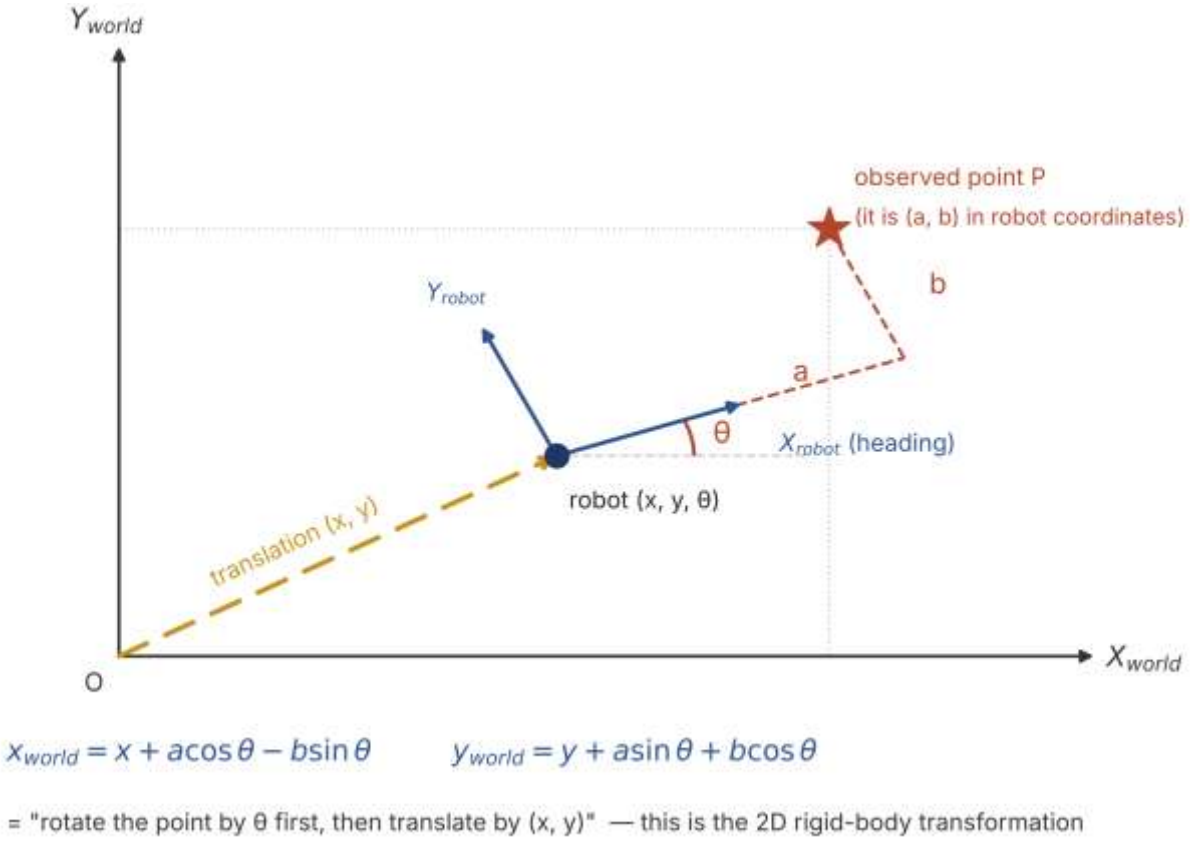
position of this point in the world frame is:

$$x_world = x + a \cdot \cos\theta - b \cdot \sin\theta$$

$$y_world = y + a \cdot \sin\theta + b \cdot \cos\theta$$

This is exactly the computation performed for every point when "plotting one revolution of LiDAR ranges into a map" in Chapter 8. Written in homogeneous coordinates, it is the 3×3 transformation matrix of the previous section times (a, b, 1). You may verify that when θ = 0 (the robot facing due east) and the pose is at the origin, the formulas reduce to x_world = a and y_world = b, as intuition requires.

Figure 10-1 2D Rigid-Body Transformation: Moving a Point from Robot Coordinates into World Coordinates



This figure is the geometric source of the most frequently used pair of expressions in the book. The robot is at (x, y) in the world frame, with its heading at angle θ to the world X axis. It measures a point with its LiDAR and records it in its own frame as (a, b)—

a ahead, b to the left. The question is: where is that point on the world map?

The answer comes in two steps. First rotate: the robot's axes are turned by θ relative to the world, so (a, b) must first be multiplied by the rotation matrix, giving ($a \cdot \cos\theta - b \cdot \sin\theta$, $a \cdot \sin\theta + b \cdot \cos\theta$). Then translate: add the robot's own position (x, y). Together these give the two expressions at the bottom of the figure.

Be sure to remember that the order is rotate first, then translate, and it cannot be reversed—a point that recurs in the ICP of Chapter 17 and the pose updates of Chapter 22. Reversing the order gives an entirely different result, and the error is often not obvious: it is barely visible while the robot is near the origin, but once it travels far the whole map goes crooked. Many students who find in Chapter 29 that "the map grows more and more skewed as the robot drives" are seeing exactly this.

An Exercise That Ties the Ideas Together

Reverse these two expressions: given a point's position in the world and the robot's pose, how would you compute its (a, b) in robot coordinates?

The answer is to multiply by the inverse transformation—and Section 10.6 will tell you that the inverse need not really be computed as a matrix inverse, since $T^{-1} = [\, R^T \quad -R^T t \,]$.

Both directions are used in SLAM: mapping uses the forward direction (moving observations into the map) and localization the reverse (moving map points into the sensor's view for comparison). Chapters 13 and 14 are exactly this pair.

Two Practical Pitfalls with Angles

- Angle wrapping: θ is periodic, and +180 degrees and −180 degrees are the same direction. When computing the difference of two angles, the result must

be wrapped back into the interval $(-\pi, \pi]$, or absurdities appear such as "turned 350 degrees" where "turned 10 degrees the other way" was meant. This is one of the most common bugs in 2D SLAM code.

- Radians and degrees: mathematical operations always use radians, while people read degrees. Units must be kept consistent in code; mixing them causes errors that are hard to notice.

10.4 The 3D Pose: Position and Attitude

UAVs, quadrupeds, and handheld cameras move in three-dimensional space and need a complete 3D pose. The 3D position is simple—just three numbers (x, y, z); what is genuinely complicated is 3D attitude: describing which way a body faces takes three rotational degrees of freedom, and there turn out to be several ways of representing those three, each with merits and drawbacks. This is the mathematical heart of the chapter.

Six Degrees of Freedom

A rigid body in three-dimensional space has six degrees of freedom (6-DoF): three translations (along X, Y, and Z) plus three rotations (about X, Y, and Z). The "6-DoF pose" that recurs in Chapters 7 and 23 means exactly these six numbers. The three positional degrees of freedom are uncontroversial, so the next three sections concentrate on the three representations of rotation.

Why is rotation so troublesome? Because a rotation is not an ordinary quantity. Two translations may be added in any order with the same result; but composing two rotations depends on the order—rotate 90 degrees about X and then 90 about Y, and the result differs entirely from rotating about Y first and then X (try it with your phone).

This non-commutativity makes the mathematics of rotation far subtler than that of translation, and it is why the MIT VNAV course of Chapter 2 emphasizes optimization on manifolds—the space of rotations is curved, not a flat vector space.

10.5 Rotation Matrices, Euler Angles, and Quaternions

This is the most important section of the chapter. 3D rotation has three principal representations, and every one of them appears in SLAM systems, so you must understand the role of each.

1. The Rotation Matrix: The Theoretical Cornerstone

A rotation is represented by a 3×3 matrix R. It rotates a vector into a new direction, and the operation is a matrix times a vector. A rotation matrix has strict mathematical properties: it is orthogonal (its columns are mutually perpendicular and of unit length) and its determinant is +1, and the set of such matrices is called the special orthogonal group SO(3).

- Advantages: mathematically perfect, directly usable in coordinate transformation, free of singularities, and composition of rotations is simply matrix multiplication.
- Drawbacks: it uses 9 numbers to represent a rotation with only 3 degrees of freedom, which is highly redundant; and in storage and optimization the orthogonality constraint must be maintained, so one cannot freely add to or subtract from the 9 numbers (or the result is no longer a legitimate rotation).

2. Euler Angles: The Most Intuitive, with a Trap

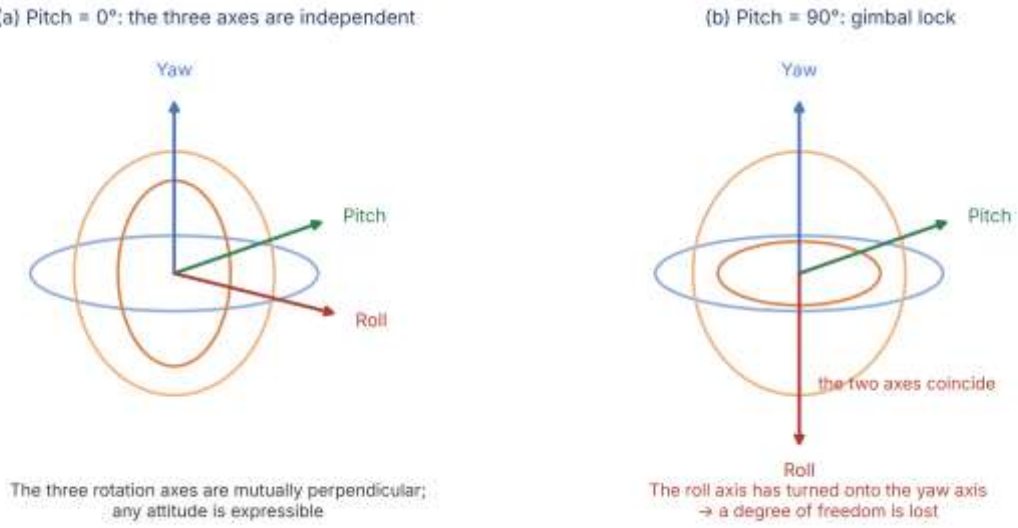
A rotation is represented by three angles—roll φ, pitch θ, and yaw ψ (introduced in Chapter 7). This is the representation closest to human intuition: "nose up 10 degrees,

banked 5 degrees right, heading east" is understood at once.

- Advantages: intuitive, using only 3 numbers (no redundancy), and convenient for people to understand and for interface display.
- Drawbacks: gimbal lock—when the pitch angle approaches ±90 degrees the roll and yaw axes coincide and a degree of freedom is lost (detailed in Chapter 7); furthermore, rotations cannot be composed by simple addition, and the same rotation may have several sets of angles (periodicity), which causes trouble in numerical optimization.

Euler angles are therefore suited to being shown to people (interface display, debugging) but not to being computed with by machines (internal computation). This is an important engineering division of labor.

Figure 10-2 Gimbal Lock: Why Euler Angles Lose a Degree of Freedom



Gimbal lock is the most notorious defect of Euler angles, yet it is not hard to understand—seeing the relationship of the three axes is enough. In the left panel the pitch angle is zero and the yaw, pitch, and roll axes are mutually perpendicular, so any attitude can be expressed uniquely by three angles.

The right panel turns the pitch angle to 90 degrees. Notice what has happened: the roll axis has been carried around by the pitch axis until it coincides with the yaw axis. Changing the yaw angle and changing the roll angle now produce exactly the same motion—the three parameters have only two effective degrees of freedom left, and a whole direction in attitude space has become inexpressible.

This is not an implementation flaw but a mathematical necessity: any method describing three-dimensional rotation with three numbers must have singularities. Avoiding them requires more than three numbers—a rotation matrix uses nine (with six constraints) and a quaternion four (with one). That is why Section 10.5 says the quaternion is the practical standard in SLAM.

The Apollo 11 navigation platform of Case 10-1 came to grief over exactly this. And on a robot it causes trouble just the same: when a UAV makes a steep dive or climb, when a robot arm's joints reach a particular attitude, or when a quadruped tips over, storing attitude as Euler angles may produce numerical blow-up or sudden attitude jumps. We shall meet it again when UAV flight control is discussed in Chapter 26.

3. The Quaternion: The Practical Standard in SLAM

A quaternion represents a rotation with four numbers (w, x, y, z) and may be understood as a compact encoding of "rotate by some angle about some axis." It is unintuitive at first sight, yet it is the rotation representation used internally and uniformly in modern SLAM, UAVs, games, and animation.

- Advantages: no gimbal lock (no singularities), only 4 numbers (minimal redundancy), composition of rotations is just quaternion multiplication (computationally efficient), numerically stable, and convenient for interpolation (turning smoothly from one attitude to another).

- Drawbacks: unintuitive, as a person can hardly read off what direction (w, x, y, z) means; and a unit-length constraint must be maintained (but only one constraint, far simpler than the orthogonality constraints of a rotation matrix).

The division of labor in practice is very clear: quaternions for internal computation and storage, Euler angles for interface display and human reading, and rotation matrices for coordinate transformation. The three interconvert, and the tf2 package in ROS and every mathematical library have conversion functions built in. You need not memorize the multiplication rules for quaternions, but you must understand why practice uses them—the answer is that they avoid both the redundancy of the rotation matrix and the gimbal lock of Euler angles.

Representation	Count of Numbers	Singularities	Principal Advantage	Principal Use
Rotation matrix	9 (3×3)	None	Direct in coordinate transformation, theoretically perfect	Coordinate transformation operations
Euler angles	3	Yes (gimbal lock)	Most intuitive, easy for people to read	Interface display, debugging
Quaternion	4	None	Compact, efficient, convenient to interpolate	Internal computation and storage

Table 10-1 Comparison of Three Representations of 3D Rotation

10.6 Coordinate Transformation and the TF Tree

With the transformation matrix as a tool, let us see how a real robotic system manages the relations among its many frames. The answer is a tree structure—the TF

tree.

Composition and Inversion of Transformations

First review two key operations, the basis on which the TF tree works:

- Composition: given the transformation from A to B and that from B to C, multiplying the two matrices gives the transformation from A to C. This lets us carry sensor data all the way into world coordinates.
- Inversion: given the transformation from A to B, its inverse matrix is the transformation from B to A.

With these two operations, knowing the transformation between any two adjacent frames suffices to compute the relation between any two frames at all—which is precisely the central idea of the TF tree.

The TF Tree: A Family Tree of Frames

On a real robot there may be a dozen or more frames: world, map, odometry, chassis, each wheel, the LiDAR, the camera, every link of a robot arm, and so on. ROS manages their relations with a tree structure (the TF tree, the transform tree).

- Each frame is a node of the tree.
- Between each parent and child node a transformation is recorded (parent to child).
- When the relation between any two frames is queried, the system automatically composes (multiplies) the transformations along the path through the tree.

The ingenuity of this design is that each transformation need only be published by the module that knows it best. The LiDAR driver publishes the fixed transformation from chassis to LiDAR (from the mounting measurements), the SLAM system publishes

the transformation from map to chassis (from the localization estimate), and odometry publishes the transformation from odom to chassis. Each module minds its own business, and the TF tree integrates them automatically into a complete network of frame relations. You will set up this tree with your own hands in the Chapter 29 exercises.

Time Stamps: A Key Detail

TF manages not only spatial relations but time. Because the robot is moving, every transformation varies with time, so the TF system stamps each transformation with a time and a query can ask "where was the camera relative to the map 0.1 second ago." This is crucial for sensor fusion—camera and LiDAR data arrive at different times and must be aligned using the transformation at the correct instant, or noticeable error appears during fast motion. This echoes the engineering problem of time synchronization in the fusion discussion of Chapter 5.

10.7 map, odom, base_link, and camera_link in ROS

ROS defines a standard naming and hierarchy of coordinate frames for mobile robots, and almost every ROS navigation system follows it. Understanding this convention is basic literacy for reading and implementing SLAM systems.

The Standard Frame Hierarchy

The standard chain, from top to bottom, is: map → odom → base_link → the individual sensor frames.

Figure 10-3 The ROS Standard Frame Hierarchy

map (the map frame): globally fixed and drift-free, but it jumps

↓ (published by the SLAM or localization system)

odom (the odometry frame): continuous and smooth, but drifts over the long run

↓ (published by odometry)

base_link (the robot body): fixed at the center of the robot's chassis

↓ (published from the fixed mounting relationship, that is, the extrinsics)

laser / camera_link / imu_link (the individual sensor frames)

Why Both map and odom Are Needed

This is the most ingenious and the most confusing part of the whole design. Why not use a single frame instead of splitting it into map and odom? Because they solve different needs and are complementary:

- odom (the odometry frame): obtained by integrating odometry and the IMU. Its character is continuous, smooth, and free of jumps—very well suited to short-term motion control and obstacle avoidance. But as Chapters 3 and 7 described, it drifts over the long run and grows less accurate the farther the robot travels.
- map (the map frame): obtained from SLAM or a localization system (AMCL, say). Its character is long-term correctness without drift, because it corrects itself by matching against the map. But it jumps: when SLAM suddenly discovers "I am actually here" and corrects the position, the robot's coordinates in map shift instantaneously.

The wisdom of keeping both layers is this: modules that need smoothness (obstacle avoidance control) use odom, and modules that need accuracy (global path planning) use map. What a SLAM system actually publishes is the map-to-odom transformation—it continually adjusts that transformation to compensate for odometry drift, while base_link's motion relative to odom remains smooth. This design lets the two apparently conflicting requirements of smoothness and accuracy coexist elegantly.

Sensor Frames and the Extrinsics

Each sensor frame below base_link (laser, camera_link, imu_link, and so on) has a fixed transformation relative to base_link, determined by where the sensor is mounted on the robot—the extrinsics mentioned in Chapter 5. These transformations are usually written in the robot's URDF description file (Chapter 29) or published by a static transform publisher. Measuring these extrinsics accurately is the basis of the system's accuracy—a mounting position off by a few centimeters or a few degrees blurs or misaligns the resulting map.

You now have the first set of mathematical tools of SLAM: how to convert poses precisely between frames. But every measurement in the real world carries noise—wheels slip, LiDAR has error, cameras mismatch features. The next chapter, "Probability, Noise, and State Estimation," teaches how to describe these uncertainties in the language of probability and make the most reasonable estimate within them. This is the pivotal turn by which SLAM moves from geometry to estimation.

Formulations and Procedures

1. The 3D Homogeneous Transformation Matrix

3D rigid-body motion is represented by a 4×4 homogeneous transformation matrix T containing both the rotation R (3×3) and the translation t (3×1):

$$T = [\,R \;\; t\,;\, 0 \;\; 1\,]$$ *(upper left 3×3 is R, upper right 3×1 is t, lower left is 0, lower right is 1)*

To convert a point p_A in frame A (in homogeneous coordinates) into frame B:

$$p_B = T_BA \cdot p_A$$

Here T_BA reads "the transformation from A to B." The order in which subscripts are written is the easiest thing to get wrong in SLAM; adopt one convention and hold to it throughout.

2. Composition and Inversion of Transformations

Composition: chain A→B with B→C into A→C:

$$T_CA = T_CB \cdot T_BA$$

Note the order of multiplication: adjacent subscripts must meet (B next to B). Inversion:

$$T_AB = (T_BA)^{-1}$$

The inverse of a homogeneous transformation matrix has a shortcut formula, with no need for a general 4×4 matrix inversion:

$$T^{-1} = [\,R^T \;\; -R^T t\,;\, 0 \;\; 1\,]$$

Exploiting the orthogonality of the rotation matrix (its inverse equals its transpose) makes inversion very efficient.

3. Composing 2D Poses (Chaining Odometry)

After the robot moves from pose $p_1 = (x_1, y_1, \theta_1)$ through a relative motion (Δx, Δy,

Δθ, expressed in the body frame of p_1), the new pose is:

$$x_2 = x_1 + \Delta x \cdot \cos\theta_1 - \Delta y \cdot \sin\theta_1$$

$$y_2 = y_1 + \Delta x \cdot \sin\theta_1 + \Delta y \cdot \cos\theta_1$$

$$\theta_2 = normalize(\theta_1 + \Delta\theta) \quad (normalize\ means\ wrapping\ back\ into\ (-\pi, \pi])$$

This is the concrete implementation of the dead reckoning of Chapter 3 and the motion update of Gmapping in Chapter 14. The normalize step is the angle wrapping emphasized in Section 10.3 and must not be omitted.

4. The Complete Pipeline by Which Sensor Data Enters a Map

Figure 10-4 The Chain of Transformations by Which One Sensor Measurement Enters the World Map

① Sensor measurement: point p in the sensor frame (camera_link, say)

↓ × T(base_link ← camera_link) the extrinsics, fixed

② Point p in the robot frame (base_link)

↓ × T(odom ← base_link) odometry

③ Point p in the odometry frame (odom)

↓ × T(map ← odom) the SLAM correction

④ Point p in the world map frame (map) → written into the map

The whole chain is a product of several transformation matrices, and the TF tree performs this computation automatically.

10.8 Contemporary Choices of Rotation Representation: From Engineering to Learning

Section 10.5 compared rotation matrices, Euler angles, and quaternions. That comparison is already complete in the context of traditional SLAM, but two recent developments deserve a supplement—one concerning engineering practice and one concerning deep learning.

1. A Quantitative Comparison: The Real Cost of Each Representation

Recent research has made a systematic comparison of seven principal representations of three-dimensional rotation—Euler angles, axis-angle vectors, quaternions, rotation matrices, the exponential map, the six-dimensional continuous representation, and probabilistic representations such as the matrix Fisher and Bingham distributions—evaluating them on continuity, gimbal lock, computational efficiency, storage, and interpolation quality.

Its conclusion agrees with Section 10.5 of this book: for general purposes the quaternion strikes the best overall balance—computationally efficient, compact in storage, smoothly interpolable by spherical linear interpolation (SLERP), and free of gimbal lock. In concrete figures, composing quaternions takes about 34 microseconds and occupies 32 bytes; rotation matrices about 306 microseconds and 72 bytes; the six-dimensional continuous representation about 422 microseconds and 48 bytes. The quaternion is thus nearly an order of magnitude faster at composition, a substantive difference in a SLAM back end that composes poses tens of thousands of times per second.

2. But Inside a Neural Network the Quaternion Is Not Best

The second development bears closely on Chapter 32. When a robot uses a neural network to predict attitude directly—regressing camera pose from an image, say, or having a vision-language-action model output a target attitude for a manipulator—the quaternion becomes a problem instead.

The reason is continuity. A neural network is trained by gradient descent and requires the output space to vary continuously with the input; but quaternions have a double-cover property (q and −q represent the same rotation), and Euler angles have the discontinuity of gimbal lock besides. These discontinuities feed the network contradictory gradients during training and make learning unstable.

The remedy is the six-dimensional continuous representation: output the first two column vectors of the rotation matrix and recover the full matrix by Gram-Schmidt orthonormalization. This representation is continuous over the space of rotations and has no singularity anywhere. The research cited above reports that in neural attitude estimation tasks it reduces error by about six to fourteen percent relative to quaternions.

This brings out an idea that matters to students: there is no best rotation representation, only the representation best for this task. A SLAM back end needs fast composition, so it uses quaternions; a neural network needs continuity and differentiability, so it uses the six-dimensional representation; people need to read it, so the user interface shows Euler angles. The same rotation puts on different clothes for different purposes.

3. Lie Groups: Making Optimization Proceed Correctly on the Manifold

There is another line, standard in modern SLAM yet often skipped in introductory texts: regard rotations and poses as Lie groups (SO(3) and SE(3)) and their small changes as

vectors in a Lie algebra.

Why is this needed? Because the optimization of Chapter 12 must differentiate with respect to pose, must compute how far two poses differ, and must add a small increment to a pose. But rotation matrices cannot simply be added—the sum of two rotation matrices is no longer a rotation matrix. The Lie algebra provides the correct procedure: add, subtract, and differentiate in the tangent space of the manifold, then send the result back to the manifold by the exponential map.

In practice you need not derive any of this from scratch—libraries such as Sophus and manif already encapsulate the operations of SO(3) and SE(3), and optimization frameworks such as g2o, Ceres, and GTSAM all have manifold parameterizations built in. Chapters 12 and 23 will use them concretely. For now, remember one sentence: a rotation is not a vector and cannot simply be added. The warning in Section 10.3 that angles cannot be averaged directly is the two-dimensional version of that same sentence.

A Quick Reference for Choosing

User interfaces, specification sheets, numbers people must read → Euler angles (but never stored internally as such)

SLAM internal state, sensor drivers, ROS messages → quaternions

Differentiation and incremental updates in optimization → Lie algebras (so(3) / se(3))

The output layer of a neural network → the six-dimensional continuous representation

Point clouds and geometric operations → rotation matrices (directly

multipliable)

Case Studies

Case 10-1 Apollo 11's Gimbal Lock Scare

Gimbal lock is not an abstract mathematical term; it once caused a real crisis during a mission to the Moon. The inertial navigation platform of the Apollo spacecraft was supported by three mechanical rings (gimbals) and tracked the spacecraft's attitude in the manner of Euler angles. When the three rings approached coplanarity, gimbal lock would occur—a rotational degree of freedom lost, and the navigation platform potentially losing its attitude reference.

The engineers knew the risk well, designed a warning indicator on the instrument panel, and planned the flight attitudes to stay clear of the dangerous region. Astronaut Michael Collins once complained half-jokingly about the restriction. In hindsight this was the inherent defect of representing attitude with Euler angles manifesting itself in an extreme mission. Modern spacecraft, UAVs, and robots long ago moved to quaternions precisely to eliminate the problem.

The lesson for students is that the choice of representation is not a matter of pure mathematical taste; it has real engineering consequences. The seemingly dry comparison of three rotation representations in Section 10.5 carries the wisdom countless engineers bought with hard experience—which is why this book insists on explaining in its mathematical chapters why a choice is made, not merely how to compute.

Case 10-2 The Frame Tree in RViz: Making the Abstract Concrete

Students meeting the TF tree for the first time often find it abstract and hard to grasp. But in RViz, the visualization tool of ROS, the frame tree becomes something visible: each frame is displayed as a little set of red, green, and blue axes (for X, Y, and Z); as the robot moves, the base_link axes move with it; and with the LiDAR mounted ahead of the robot, the laser axes stay fixed a short distance in front of base_link.

More instructive still is observing errors. If the extrinsics are set wrong (the LiDAR's mounting direction entered backwards, say), the point cloud of the LiDAR scan in RViz will be displaced from the real environment—the wall is clearly ahead, yet the points are drawn behind or to the side. By adjusting the extrinsics and watching in real time whether the point cloud hugs the environment, students understand the meaning of coordinate transformation very directly. This is the best experiment for threading every abstract concept of Sections 10.1 through 10.7 together at once, and the Chapter 29 exercises use it extensively.

A common debugging maxim in university classrooms is: "if the map is crooked, look at TF first." Many SLAM mapping failures have their root not in the algorithm but in misconfigured frames—extrinsics entered wrongly, time stamps unaligned, a broken TF tree. Understanding this chapter saves students countless wasted hours of debugging.

Case 10-3 Forward Kinematics of a Robot Arm: The Same Mathematics on Another Stage

The coordinate transformation mathematics of this chapter is not used only in mobile robots; it is also the core of robot arms. A six-axis arm has six joints between base and gripper, and the rotation of each joint is a rotational transformation. Multiplying the transformation matrices in order—base to first link, first link to second,

and so on to the gripper—computes the pose of the gripper in space given the six joint angles, which is called forward kinematics.

This is a direct application of "composing transformations is just multiplying matrices" from Section 10.6. Students taking a robot arm course alongside this one will find that both use the same homogeneous transformation mathematics—no coincidence, but because "how a rigid body moves in space" is the common language of robotics. Mastering this chapter lays the mathematical foundation for mobile robots and robot arms alike. This also echoes the shared sensor → mathematics → systems architecture of the university courses in Chapter 2: mathematics is the common denominator of all robotics.

Hands-On Activities

Exercise 10-1 Working Coordinate Transformations by Hand and Observing the TF Tree

Purpose: to master 2D coordinate transformation through pencil-and-paper computation and verification in code, and to observe a real TF tree in ROS so that the abstract concepts become concrete. This exercise has three parts—paper, code, and ROS—and suits teaching at various depths.

Part One: Pencil and Paper (everyone)

1. A robot has pose ($x = 3$, $y = 2$, $\theta = 90°$) in the world frame. Its LiDAR measures a point in the body frame ($a = 2$ ahead, $b = 0$ to the left). Compute this point's world coordinates by hand with the formulas of Section 10.3, and draw the figure on graph paper to check that the answer is sensible.

2. For the same robot, the LiDAR measures another point (a = 1, b = 1). Compute its world coordinates by hand and confirm with a drawing.
3. Angle wrapping exercise: the robot's heading is $\theta_1 = 170°$ and it turns right by $\Delta\theta = 30°$. Direct addition gives 200°; what should it be after wrapping back into $(-180°, 180°]$? Explain why not handling this causes trouble.

Part Two: Verification in Python (everyone)

4. Build the 3×3 homogeneous transformation matrix of Section 10.2 with numpy, redo the computation of Part One as a matrix multiplication, and verify that the results agree.
5. Implement composition of transformations: given the world→robot and robot→LiDAR transformations, multiply them to obtain world→LiDAR, and verify.
6. Implement inversion: take the inverse of the world→robot transformation and verify that it equals the robot→world transformation (compare the shortcut formula of Section 10.2 with a direct numpy inverse).
7. (Undergraduate) Implement the mutual conversion of Euler angles, rotation matrices, and quaternions (scipy.spatial.transform.Rotation may be used); take an attitude with pitch near 90° and observe how the Euler angle representation becomes anomalous, verifying gimbal lock first-hand.

Part Three: Observing the ROS TF Tree (for those with a ROS environment)

8. Launch the TurtleBot3 simulation (Chapter 28) or any ROS system with TF, use view_frames or rqt_tf_tree to produce a diagram of the TF tree, and identify the hierarchical relations among map, odom, base_link, and the sensor frames.

9. Display TF in RViz, move the robot, and observe which axes move with it (base_link and its child frames) and which do not (map).
10. Use the tf2 query tools to look up the transformation of camera_link relative to map, querying once before and once after the robot moves, and observe how the values change.

Discussion Questions

1. In Part One, when the robot's heading is 90° (due north), which direction in the world frame does the body's "ahead" correspond to? Does your computed result match intuition?
2. Why does a SLAM system publish the map→odom transformation rather than map→base_link directly? How does this design help reconcile smoothness with accuracy?
3. (Undergraduate) What anomaly did you observe in the gimbal lock experiment? If a UAV's flight controller used Euler angles rather than quaternions, what might happen during an aerobatic roll?
4. If the map a robot builds is always skewed, or the point cloud does not line up with the walls, what would you check first based on this chapter?

Extended Challenge (Undergraduate)

Write a ROS 2 node that publishes a static transformation of a custom sensor relative to base_link (deliberately setting a known rotation and translation), display that sensor's simulated point cloud in RViz, and adjust the extrinsic values while watching in real time how the cloud moves, until it sits where expected. This exercise corresponds directly to configuring a robot's TF in the Chapter 29 exercises and lays the coordinate groundwork for the 2D SLAM vehicle implementation.

Further Reading

1. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Lecture 3 (rigid-body motion in three-dimensional space) and Lecture 4 (Lie groups and Lie algebras), Publishing House of Electronics Industry. (The clearest introduction to the rotation mathematics of SLAM in the Chinese-speaking world.)
2. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017. (The authoritative textbook on rigid-body motion and coordinate transformation, with a free electronic edition and online course.)
3. J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," arXiv, 2018. (A friendly introduction to rotation groups and optimization on manifolds, leading into Chapter 12.)
4. The official ROS tf2 tutorials (docs.ros.org). (Implementation guidance for the TF tree, with complete examples.)
5. REP 105: Coordinate Frames for Mobile Platforms (ros.org/reps). (The official definition of the ROS standard frames map, odom, and base_link.)
6. 3Blue1Brown, the "Quaternions and 3D rotation" video series (YouTube). (Explains quaternions intuitively with superb animation; strongly recommended.)
7. R. Siegwart et al., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (Chapters 2 and 3 place frames and kinematics in their robotics context.)

From Certainty to Uncertainty: Where This Chapter Stands

This chapter has accomplished one thing: making the data of all sensors expressible in

one frame. But it has quietly assumed a premise that does not hold in the real world—that every number is exact.

The expressions of Figure 10-1 assume the robot's pose (x, y, θ) is known and correct. But Chapter 7 told us the IMU drifts, Chapter 5 told us GNSS error is amplified by DOP, and the wheel odometry of Chapter 13 slips. In reality a robot never knows exactly where it is—it has only an estimate carrying an uncertainty.

Likewise, the observed point P in Figure 10-1 is not an exact point either. The LiDAR of Chapter 8 has ranging error and limited angular resolution, and the camera of Chapter 9 has pixel quantization and feature localization error. Every observation is a blur.

So Chapter 11 replaces every equals sign of this chapter with a probability distribution: a pose is no longer three numbers but a Gaussian distribution; an observation is no longer a point but a measurement with a covariance. And the work of fusing several sensors becomes "multiply several distributions together and find the most probable answer." That is Bayesian filtering, the common skeleton of the Kalman filter and the particle filter. The transformation matrices of this chapter do not disappear—they become the motion model and the observation model within the probabilistic framework.

Summary

1. SLAM handles three levels of frame at once—world, robot, and sensor; the sensor mounting relationship is fixed (the extrinsics) while the robot pose varies (the quantity SLAM estimates), and a piece of data entering the map passes through the chain sensor → robot → world.

2. Homogeneous coordinates add one dimension to unify rotation (multiplication) and translation (addition) into a single matrix multiplication; 2D uses a 3×3 and 3D a 4×4 transformation matrix, so that chaining transformations becomes a product and inversion becomes taking an inverse matrix.
3. A 2D pose is described by the three numbers (x, y, θ), with coordinate transformation given by formulas containing cosθ and sinθ; in practice angle wrapping (back into −π to π) must be handled and radians and degrees kept consistent.
4. A 3D rigid body has six degrees of freedom (three translational, three rotational); composition of rotations is non-commutative and the space they form is curved, which is the root of the complexity of rotation mathematics and of optimization on manifolds.
5. The three representations of 3D rotation have their own roles: the rotation matrix (9 numbers, no singularity, used in coordinate transformation), Euler angles (3 numbers, intuitive but with gimbal lock, used in human interfaces), and the quaternion (4 numbers, no singularity, efficient, used for internal computation and storage).
6. The TF tree manages many frames in a tree structure, each transformation published by the module that knows it best, with transformations composed automatically along the path on query; transformations carry time stamps to support temporal alignment in sensor fusion.
7. The ROS standard frames run map → odom → base_link → sensors: odom is continuous and smooth but drifts, while map is accurate but jumps; SLAM

publishes the map→odom transformation to compensate for drift, letting smoothness and accuracy coexist elegantly.

This chapter has assembled the first set of mathematical tools of SLAM—deterministic coordinate transformation. But the real world is full of uncertainty: odometry slips, LiDAR has stray points, cameras mismatch features. The next chapter, "Probability, Noise, and State Estimation," introduces the language of probability and teaches a robot to say "I am probably here, but I am not quite sure" and to make the most reasonable estimate amid noise. This is the pivotal step by which SLAM moves from geometry to estimation, and the common starting point of Bayesian filtering, the Kalman filter, and the particle filter.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 11

Probability, Noise, and State Estimation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 10 made the data of all sensors expressible in one frame, but it assumed every number was exact. This chapter dismantles that assumption.

A real robot never knows exactly where it is. Wheels slip, the IMU drifts, the LiDAR has ranging error, and the camera localizes feature points imprecisely. More troublesome still, these errors contradict one another—odometry says it traveled three meters while LiDAR matching says only two point seven. Which should be believed?

The answer this chapter gives is: both, weighted by how far each can be trusted. And for trustworthiness to be computable, every estimate must first be written as a probability distribution. So a pose is no longer three numbers but a Gaussian distribution; an observation is no longer a point but a measurement with a covariance; and fusion is no longer averaging but multiplying distributions.

This is the most crucial conceptual turn in the book. The optimization of Chapter 12, Gmapping in Chapter 14, Cartographer in Chapter 15, and ORB-SLAM3 in Chapter 23 all rest on the framework of this chapter—they differ only in how they represent that distribution and how they find the most probable answer.

Figure 11-0 How Chapter 11 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 10 A pose = three (or six) exact numbers

This chapter A pose = a probability distribution carrying an uncertainty

[This chapter is responsible for]

The Gaussian distribution and covariance: describing "roughly where I am" with a mean plus

an ellipse

The motion model and the observation model: the two sources of prediction and correction

Bayesian filtering → Kalman → EKF → particle filter

[Where it leads]

→ Chapter 12 Another way to solve the same problem: optimization (the counterpart to filtering)

→ Chapter 13 Every cell of an occupancy grid map is a probability too

→ Chapter 14 Gmapping = a particle filter + a map carried by every particle

→ Chapter 16 AMCL: relocalization in a cleaning robot is the MCL of this chapter

→ Chapter 23 The MAP estimation of ORB-SLAM3 is the optimization version of this chapter's ideas

Learning Objectives

In the previous chapter we learned to convert precisely between frames—that was deterministic mathematics. But the real world is not so clean: odometry slips, LiDAR has stray points, cameras mismatch features. This chapter introduces the language of probability and teaches a robot to say "I am probably here, but I am not quite sure" and to make the most reasonable estimate amid noise. This is the pivotal turn by which SLAM moves from geometry to estimation, and the mathematical core of the book. After completing this chapter, you should be able to:

1. Explain why SLAM must handle uncertainty probabilistically rather than pursue a single exact answer.
2. Explain the meaning of the mean and variance of a Gaussian distribution, and understand how covariance describes uncertainty in two or more dimensions.

3. Explain how the motion model and the observation model describe a robot's prediction and perception probabilistically.
4. Describe the recursive predict-update structure of Bayesian filtering and understand that it is the common skeleton of every filter.
5. Describe how the Kalman filter works and explain how the Kalman gain balances prediction against observation.
6. Explain how the EKF handles nonlinearity, together with the basic concept and limitations of EKF-SLAM.
7. Explain the principle of the particle filter and Monte Carlo localization, and compare it with the Kalman family.

Core Concepts

Chinese Term	English Term	Explanation
機率分布	Probability Distribution	Describes the probability of each possible value of a random variable.
高斯分布	Gaussian Distribution	The bell-shaped normal distribution, described completely by a mean and a variance.
均值	Mean	The center of a distribution, representing the most likely estimate.
變異數	Variance	The spread of a distribution, representing the magnitude of the uncertainty.
共變異數	Covariance	Describes the correlation and joint uncertainty among multidimensional random variables.
運動模型	Motion Model	Describes how the state changes after the robot executes a control (noise included).

Chinese Term	English Term	Explanation
觀測模型	Observation Model	Describes what the sensor would measure in a given state (noise included).
貝氏濾波	Bayes Filter	The general framework estimating state recursively through the two steps of prediction and update.
卡爾曼濾波	Kalman Filter	The optimal recursive estimator for a linear Gaussian system.
卡爾曼增益	Kalman Gain	The weight determining whether to believe the prediction or the observation.
擴展卡爾曼濾波	EKF	Applies the Kalman filter after linearizing a nonlinear system.
粒子濾波	Particle Filter	A nonparametric filter approximating an arbitrary distribution with a large number of samples.
蒙地卡羅定位	MCL	Localization within a known map by particle filter—that is, AMCL.

11.1 Why SLAM Must Handle Uncertainty

Recall the striking experiment of Chapter 7: a device completely at rest, whose "position" computed by integrating the IMU drifted by several meters within 60 seconds. Think too of the Wi-Fi positioning of Chapter 4, the GPS jumps of Chapter 5, and the camera mismatches of Chapter 9—not one sensor gives a true value; what each gives is a guess carrying noise.

There are two attitudes one may take toward this reality. The naive approach is to pretend the sensors are accurate and use their readings directly. That is exactly why dead reckoning fails—take every erroneous measurement for truth and accumulate it, and the error accumulates without end.

The mature approach is to admit that every measurement is uncertain, quantify that uncertainty, and then strike the best balance among several uncertain pieces of information. This is the central idea of probabilistic robotics and the mathematical backbone of modern SLAM.

The Key Idea: What Is Estimated Is a Distribution, Not a Value

This is the most important conceptual shift in the chapter. Traditional thinking says "the robot is at (3.5, 2.1)"; probabilistic thinking says "the robot is most likely near (3.5, 2.1), but it may also be anywhere within an ellipse around it, with likelihood increasing toward the center."

In other words, what SLAM estimates is not a point but a whole probability distribution—it tells us both the best guess and how trustworthy that guess is. That trustworthiness is crucial, because it determines how far the next step should believe this estimate. The probabilistic definition of SLAM given in Chapter 1, p(x, m | z, u), is

precisely saying "what the probability distribution of the state looks like, given all observations and controls."

An Intuitive Analogy: Walking Blindfolded

Imagine walking blindfolded through a familiar room. With each step you roughly know how far you have advanced (motion),

but not exactly—perhaps a little more, perhaps a little less (motion noise). Your grasp of your own position grows vaguer and vaguer.

Then your hand touches the corner of a wall (an observation). That definite landmark makes your grasp of your position sharp at once.

SLAM is doing exactly this: uncertainty enlarges while walking (prediction) and contracts on touching a landmark (update).

What this chapter teaches is how to describe these two actions—blurring and sharpening—precisely in mathematics.

11.2 The Gaussian Distribution: Mean, Variance, and Covariance

Describing uncertainty mathematically requires a distribution. The one most used in SLAM is the Gaussian (or normal) distribution, because it has two irreplaceable advantages: it is completely described by only two parameters, and it remains Gaussian under a variety of operations (which is mathematically very friendly).

The One-Dimensional Gaussian: A Bell Curve

The one-dimensional Gaussian is the familiar bell curve, determined completely by two parameters:

- The mean μ (mu): the center of the curve, representing the most likely value—our best estimate.
- The variance σ^2 (sigma squared): how fat or thin the curve is, representing the magnitude of the uncertainty. A small variance gives a tall, narrow curve, meaning high confidence; a large variance gives a short, wide one, meaning great uncertainty.

The standard deviation σ is the square root of the variance and shares units with the measurement, which is more intuitive. "The robot is at 3.5 meters with a standard deviation of 0.2 meters" means that the best estimate is 3.5 and the truth falls between 3.3 and 3.7 with about 68% probability (the one-standard-deviation range of a Gaussian).

The Multivariate Gaussian: From a Point to an Ellipse

A robot's position is two-dimensional (x, y), so the uncertainty becomes two-dimensional as well. The mean is then a point (the most likely position), while the uncertainty is described by a covariance matrix.

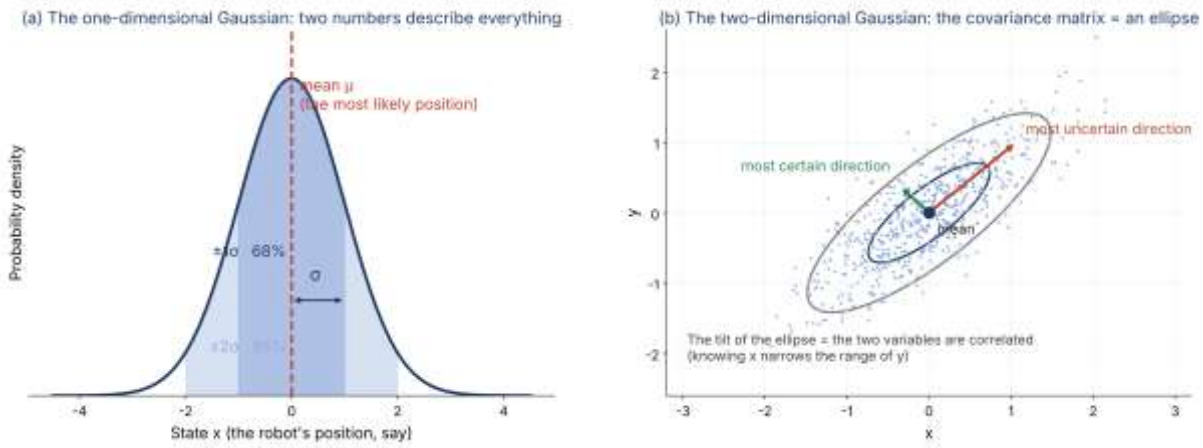
The geometric meaning of the covariance matrix is very intuitive: it corresponds to an uncertainty ellipse. The ellipse's center is the mean, its size reflects how uncertain the estimate is, and its shape and orientation reflect which directions are more uncertain.

- Diagonal elements: the variance along each axis (how uncertain in x, how uncertain in y).

- Off-diagonal elements (the covariances): the correlation between x and y. A positive value means "when x runs large, y tends to run large too," and the ellipse tilts.

Take an example from SLAM: a robot driving straight along a corridor is very uncertain about how far it has traveled (along the corridor, where error accumulates) but quite certain about whether it has strayed from the centerline (perpendicular, where the walls keep correcting it). Its uncertainty ellipse is therefore a long, narrow one stretched along the corridor. This ellipse keeps deforming as the robot moves and observes—every step of SLAM is, in essence, updating this ellipse.

Figure 11-1 The Gaussian Distribution: From a One-Dimensional Bell Curve to a Two-Dimensional Uncertainty Ellipse



The left panel is the one-dimensional case. The whole curve is determined by just two numbers: the mean μ (the most likely position) and the standard deviation σ (how uncertain it is). 68% of the probability falls within ±1σ and 95% within ±2σ—two numbers worth memorizing, because judging in practice whether an estimate is accurate enough uses them constantly.

The right panel is the two-dimensional case, and that ellipse recurs throughout the rest of this book. Note that it is tilted—which means the errors in x and y are correlated. The

red arrow points along the ellipse's major axis, the most uncertain direction; the green arrow along the minor axis, the most certain.

This tilt has a concrete physical meaning in SLAM. As a robot advances along a corridor, its lateral position can be locked precisely by the walls on either side, while its longitudinal position can only accumulate from odometry—so the uncertainty ellipse is stretched along the corridor. The degeneracy of Section 17.5 and the "distant points measure poorly" of Chapter 21 both draw this same elongated ellipse.

Remember one thing in particular: a covariance matrix is not a heap of abstract numbers, it is this ellipse. The diagonal elements set its size along each axis and the off-diagonal elements set how far it tilts. When you see the symbols P and Σ hereafter, an ellipse should come to mind at once.

Why the Gaussian Is Preferred

- Two parameters suffice: mean and covariance describe the uncertainty completely, and both storage and computation are light.
- Closure under operations: the product of two Gaussians is Gaussian, and a Gaussian under a linear transformation remains Gaussian. This lets every step of a filter stay Gaussian rather than growing ever more complicated, and it is the mathematical root of the Kalman filter's elegance.
- The central limit theorem: many independent small errors superimposed tend toward a Gaussian. Sensor noise is often precisely the sum of many tiny factors, so the Gaussian assumption often holds in practice.

A caution, however: the Gaussian assumption is not universal. When the uncertainty is multimodal (the robot may be in one of two identical-looking rooms but does not know which), a single Gaussian cannot describe it—which is exactly why the

particle filter enters in Section 11.7.

11.3 Sensor Noise and the Motion Model

Placing a robot in a probabilistic framework requires two models, which are the inputs to every filter. The two describe how the robot moves and how it sees.

1. The Motion Model

The motion model answers: "after the robot executes some control (advance one meter, turn 30 degrees left), what will the new pose be?" The key is that the answer is not a definite pose but a probability distribution, because executing a control carries noise (wheel slip, uneven ground).

Mathematically it is written p(x_t | x_{t−1}, u_t): the probability distribution of the new pose x_t given the previous pose x_{t−1} and the control u_t. The effect of the motion model is to enlarge the uncertainty—every step inflates the uncertainty ellipse a little, because fresh motion noise has accumulated. This is exactly the 2D pose composition formula of Chapter 10 with a noise term added.

2. The Observation Model

The observation model answers: "if the robot really is at some pose, what should the sensor measure?" Here too the answer is a probability distribution, because the sensor is noisy.

Mathematically it is written p(z_t | x_t): the probability distribution of the observation z_t given the pose x_t. For instance, "if the robot is here, the LiDAR should measure a wall 2 meters ahead," but the actual measurement will jitter around 2 meters. The effect of the observation model is to contract the uncertainty—a definite observation (touching a corner) greatly narrows the range of possible positions.

The Division of Labor Between the Two Models

These two models correspond exactly to the two actions in the blindfolded-walking analogy of Section 11.1: the motion model enlarges the uncertainty (the longer you walk, the vaguer it gets), and the observation model contracts it (touching a landmark makes it sharp again). The whole process of SLAM is these two models alternating—move a step, take a look, move a step, take a look. The Bayesian filter of the next section expresses that alternation precisely in mathematics.

	Motion Model	Observation Model
Mathematical form	p(x_t | x_{t−1}, u_t)	p(z_t | x_t)
Question answered	Where it will be after moving	What would be seen from here
Inputs	The previous pose + the control command	The current pose
Source of noise	Slip, terrain, wheel diameter error	Sensor measurement error
Effect on uncertainty	Enlarges (inflates)	Contracts (shrinks)
Corresponding analogy	Walking blindfolded	Touching a wall corner

Table 11-1 The Motion Model and the Observation Model Compared

11.4 The Bayes Filter

The Bayes filter is the core of this chapter—it is the common skeleton of the Kalman filter, the EKF, the particle filter, and every other filter. Once it is understood, the sections that follow are merely different implementations of it.

Belief: A Probabilistic Description of the State

The central object a Bayes filter tracks is called the belief, written bel(x_t)—the probability distribution of the current state given all information so far. The belief is the mathematical substance of the uncertainty ellipse described in Section 11.1. The task of the Bayes filter is to update that belief whenever a new piece of data arrives.

A Recursion of Two Steps

The Bayes filter cycles through two steps, corresponding exactly to the two models of the previous section:

1. Prediction: the robot executes control u_t, and the motion model pushes the belief forward. This step enlarges the uncertainty—"I moved, so I am less sure where I am." Mathematically it integrates the previous belief against the motion model (summing over all possible previous poses).
2. Update (correction): the sensor yields observation z_t, and the observation model corrects the belief. This step contracts the uncertainty—"I saw a landmark, so I am surer where I am." Mathematically it multiplies the predicted belief by the observation model and normalizes.

This predict-update cycle is the very same thing as the standard SLAM pipeline of Figure 1-3 in Chapter 1 and the predict-update of the Kalman filter in Chapter 7, now given a precise probabilistic statement. Fix this two-step structure firmly in mind: it is the key to understanding every recursive estimator.

Figure 11-2 The Recursive Cycle of Bayesian Filtering

The previous belief bel(x_{t−1})

↓ ① Prediction: apply the motion model p(x_t | x_{t−1}, u_t)

→ the belief widens (uncertainty enlarges)

The predicted belief (the prior)

↓ ② Update: multiply by the observation model p(z_t | x_t) and normalize

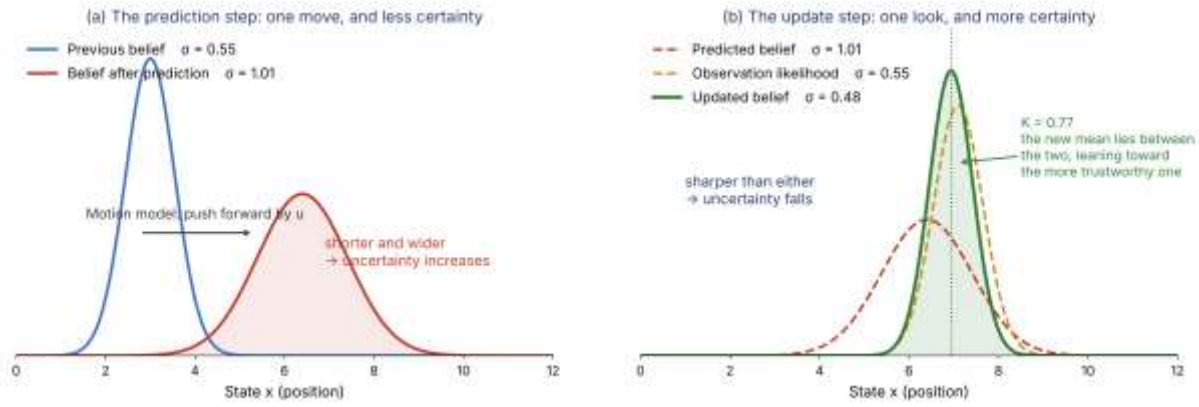
→ the belief narrows (uncertainty contracts)

The new belief bel(x_t)

↓

(back to the top for the next instant)

Figure 11-3 The Two Steps of Bayesian Filtering: Prediction Widens the Distribution, Update Sharpens It



This figure is the one most worth understanding in the chapter—arguably in the whole book. It lays out the entire Bayesian filtering cycle in a one-dimensional example.

The left panel is the prediction step. The robot was fairly sure it was at position 3 (σ = 0.55), and then odometry says "moved forward by 3.4." The mean of the belief moves forward to 6.4, but the curve also becomes shorter and wider (σ grows to 1.01)—

because motion itself carries noise, and the farther one travels the less certain one becomes. This is the integration drift of Chapter 7 and the odometry error of Chapter 13 expressed in probabilistic language: with dead reckoning alone, uncertainty can only rise monotonically.

The right panel is the update step. The sensor now reports an observation placing the position near 7.1 ($\sigma = 0.55$). Multiplying the predicted belief by the observation likelihood and normalizing gives a posterior belief lying between the two (6.94) and sharper than either (σ falls to 0.48).

Where between the two it lands is set by the Kalman gain. In this example $K = 0.77$, meaning the new mean has moved 77% of the way toward the observation—because the observation ($\sigma = 0.55$) is more trustworthy than the prediction ($\sigma = 1.01$). If the observation noise were large, K would approach 0 and the robot would nearly ignore the observation; conversely, if the observation were extremely accurate, K would approach 1 and the robot would nearly accept it outright. Section 11.5 gives the full formula for K, but its meaning is in this figure.

Three Conclusions Worth Remembering

1. The uncertainty after an update is always smaller than before it. This is the fundamental value of fusion—one more independent observation, one degree less uncertainty.

2. The fused mean always lies between the two sources and never outside that range. So a Kalman filter can never be "more absurd than either source."

3. The whole cycle has only these two steps: prediction widens, update sharpens. The particle filter of Chapter 14, the scan matching of Chapter 15,

and the visual-inertial optimization of Chapter 23 are all doing the same thing in different representations.

Bayes' Rule: The Basis of the Update Step

The mathematics behind the update step is the famous Bayes' rule of probability theory. Its spirit is to combine the prior belief (what the prediction step produced, before the observation was seen) with the evidence the observation provides, giving the posterior belief (a more precise belief after the observation).

$$\textit{posterior} \propto \textit{observation likelihood} \times \textit{prior belief} \qquad bel(x_t) \propto p(z_t \mid x_t) \cdot \textit{predicted}$$

This expression reads: the updated belief is proportional to the observation model times the predicted belief. It is the theoretical root of the update step of every filter. The Bayes filter is itself an abstract framework; in actual computation arbitrary distributions cannot be handled directly—the different filters are different answers to the questions of what form the belief takes and how the integral and the product are computed. The next three sections give three such answers.

11.5 The Kalman Filter

The Kalman filter (KF) is the exact solution of the Bayes filter under the assumption of a linear system with Gaussian noise. Under that assumption the belief is always Gaussian, so the whole filter need track only two quantities: the mean (the best estimate) and the covariance (the uncertainty). This makes it extremely efficient and has made the Kalman filter one of the most successful algorithms in engineering history—from Apollo lunar navigation to the phone in your pocket, it is everywhere.

The Two Steps of the Kalman Filter

It follows the Bayes filter architecture exactly, only with every operation carried out in the world of Gaussians:

3. Prediction: push the mean forward with the motion model (the robot moved, so the estimated position moves with it) and enlarge the covariance (adding motion noise, so uncertainty grows).
4. Update: correct the mean with the observation and shrink the covariance. The strength of the correction is set by the Kalman gain.

The Kalman Gain: The Most Ingenious Concept in the Chapter

The Kalman gain (K) is the soul of the filter. It answers a key question: when the prediction and the observation disagree, which should be believed, and by how much?

Its logic is entirely intuitive:

- If the prediction is very uncertain (large covariance) and the observation accurate (small noise): the gain is large and the observation is believed more—"I was not sure anyway, and since the sensor is so definite, let us go with it."
- If the prediction is confident and the observation noisy: the gain is small and the prediction is believed more—"I am quite sure of my own dead reckoning; this very noisy measurement is just a reference."

In other words, the Kalman filter does not stolidly take an average but weights by the trustworthiness of each side—whichever is more trustworthy is listened to more. This idea of weighting by trustworthiness is the mathematical embodiment of the fusion spirit that recurs throughout this book (Chapters 4, 5, and 7). The attitude filtering of Chapter 7 and the GNSS/INS integrated navigation of Chapter 5 both have this gain working underneath.

A Summary of the Idea: What a Kalman Filter Does

In one sentence: predict the present from the previous estimate plus the motion, correct it with the sensor observation,

and fuse the two weighted by their respective trustworthiness (covariance) to obtain an estimate more accurate than either alone.

The key insight: fusing two noisy pieces of information yields a result more accurate than either one by itself—

which is why multi-sensor fusion always beats using only the best single sensor.

11.6 The Extended Kalman Filter and EKF-SLAM

The Kalman filter is beautiful but has a strict premise: the system must be linear. Unfortunately the robot's world is full of nonlinearity—as Chapter 10 showed, a pose update contains cosθ and sinθ, and the observation model "how far would the LiDAR measure if I were here" is nonlinear too. The standard Kalman filter cannot handle this directly. The EKF (extended Kalman filter) was created for exactly this.

The EKF's Central Trick: Local Linearization

The EKF's idea is very pragmatic: since the system is nonlinear, approximate the curve by its tangent near the current estimate. Mathematically this is a first-order Taylor expansion, and the resulting matrix of slopes is called the Jacobian. Once linearized, the standard Kalman filter formulas apply.

This approximation works well when the nonlinearity is not too severe and the uncertainty not too large, and it is the most widely used estimation method in robotics—the UAV flight-control EKF mentioned in Chapter 7 and the robot_localization package in ROS both rest on it. But its approximate nature brings limits too: when the nonlinearity is strong or the uncertainty large, the tangent departs too far from the curve and the estimate may lose accuracy or even diverge.

EKF-SLAM: Putting the Map into the State as Well

How is the EKF used for SLAM? The key step is to put the robot's pose and the positions of all landmarks into one state vector and estimate them together with a single Gaussian distribution (a single large covariance matrix).

This design captures the essence of SLAM ingeniously—the coupling of localization and mapping emphasized in Chapter 1. When the robot observes a landmark and corrects its own position, the correlations in the covariance matrix propagate that correction to all related landmarks and poses. This is precisely the mathematical mechanism by which a loop closure flattens the error of a whole map at once: on returning to a familiar place, one strong observation corrects the robot and every landmark along the way together, through the correlations in the covariance.

The Historical Standing and the Limitations of EKF-SLAM

EKF-SLAM was the first complete and workable solution in the history of SLAM (in the 1990s), laid the theoretical foundation of the whole field, and holds an honored place. But it has two fatal limitations that gradually pushed it out of the mainstream:

- Computational complexity: the size of the covariance matrix grows as the square of the number of landmarks. With many landmarks (thousands), the

matrix operations become too slow for real time—a hard limit for large scenes with tens of thousands of landmarks.

- Accumulating linearization error: the EKF makes a linear approximation at every step, and the errors accumulate, which over long runs or large scenes may make the estimate inconsistent.

These two limitations gave rise to the two mainstream lines that followed: the particle filter of Chapter 14 (handling it another way) and graph optimization in Chapter 12 (replacing recursive filtering with batch optimization). One may say that understanding the limitations of EKF-SLAM is understanding why modern SLAM turned to graph optimization—which makes this a pivotal chapter between what came before and what comes after.

11.7 The Particle Filter and Monte Carlo Localization

The whole Kalman family assumes the belief is a single Gaussian. But as the end of Section 11.2 noted, in some situations the belief is multimodal—the robot may be in one of two identical rooms, and the belief should have two peaks. A single Gaussian cannot describe this. The particle filter offers an entirely different approach.

Representing a Belief with a Crowd of Samples

The central insight of the particle filter is this: rather than describing the distribution with a mathematical formula, approximate it with a large crowd of samples (particles). Each particle represents one hypothesis that "the robot might be here." Where the particles are dense, the likelihood is high.

The greatest advantage of this representation is that it can describe a distribution of any shape—multimodal, irregular, non-Gaussian, all are fine. This the Kalman family

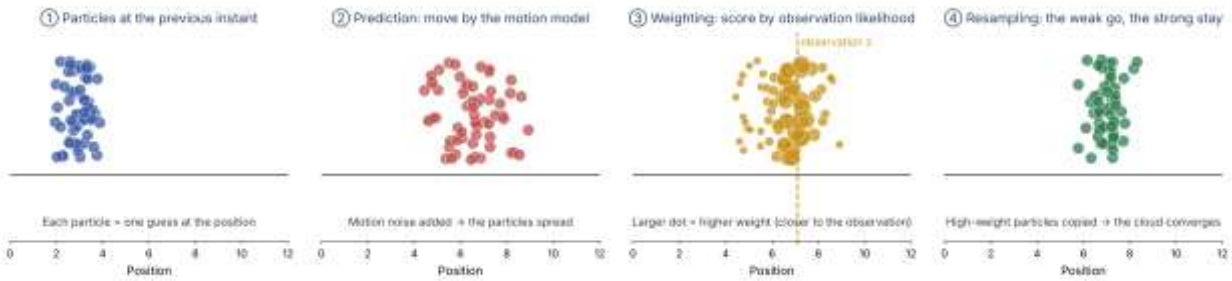
cannot do.

The Three Steps of the Particle Filter

It too follows the Bayes filter architecture, but realized with particles:

5. Prediction (move the particles): each particle moves according to the motion model, with its own random noise added. The once-clustered cloud of particles spreads out—corresponding to the enlargement of uncertainty.
6. Update (weigh them): the observation model scores each particle (assigns a weight). Particles that agree with the observation (the wall expected at this position matches what was measured) score high, and those that do not score low.
7. Resampling: particles are drawn afresh according to their weights—high-scoring particles are copied several times and low-scoring ones are eliminated. The cloud gradually gathers at the most likely position. This is the essence of the particle filter: survival of the fittest.

Figure 11-4 The Four Steps of the Particle Filter: Representing a Probability Distribution with a Crowd of Samples



The particle filter abandons describing a distribution with a formula and represents it with a crowd of samples instead. Every particle is a complete guess at the state—in the localization problem, a hypothesis that "the robot might be here, facing this way." Where the particles are dense, the probability is high.

The four steps correspond exactly to the two steps of Figure 11-3. ① is the particle cloud at the previous instant. ② Prediction: each particle moves forward on its own according to the motion model with random noise added—the result is that the particles spread, corresponding to the "widening" in the Gaussian version. ③ Weighting: each particle is scored by the observation likelihood, and those agreeing with the observation receive high weight (drawn as larger dots). ④ Resampling: particles are drawn anew by weight, high-weight particles copied several times and low-weight ones eliminated, so the cloud converges—corresponding to the "sharpening" in the Gaussian version.

What, then, makes it better than the Kalman filter? The key is that the particle cloud may take any shape. The Kalman filter is locked into a unimodal Gaussian, while real localization problems are often multimodal: a robot at power-on may be anywhere in the building; walking a symmetric corridor, the observations at two positions look identical. In such situations "a mean plus an ellipse" simply cannot express the belief, while a particle cloud can keep a group of particles in each of two places and decide which to keep once more observations arrive.

The price is computation. The number of particles must grow exponentially with the dimension of the state, so the particle filter is very practical for 2D localization (three degrees of freedom) but cannot cope in high dimensions. That is exactly why Gmapping in Chapter 14 must adopt the Rao-Blackwellization trick—particles only for the pose, with the map handled analytically.

Monte Carlo Localization (MCL) and AMCL

Applying the particle filter to localization within a known map gives Monte Carlo localization (MCL), whose implementation in ROS is called AMCL (mentioned in

Chapter 3). Its operation is extremely intuitive: not knowing where the robot is at first, scatter particles everywhere on the map; as the robot moves and scans, particles inconsistent with the observations are eliminated and the cloud gradually converges on the true position.

MCL inherently solves the kidnapped robot problem of Chapter 3: even if the robot is carried away, so long as some particles remain scattered across the map they can win out at the new position and let the robot find itself again. This is hard for a single-Gaussian Kalman filter to do—and it is the mathematical reason a cleaning robot can relocalize after being carried into another room.

The Particle Filter and SLAM: A Preview of Gmapping

Applying the particle filter to SLAM (rather than localization alone) requires an ingenious trick called the Rao-Blackwellized particle filter (RBPF)—each particle represents not only a pose but carries a whole map of its own. This is the core of Gmapping in Chapter 14 and the protagonist of the 2D SLAM of Part IV. This chapter sets out the principle of the particle filter clearly so that Chapter 14 can concentrate on how it builds maps.

Characteristic	The Kalman / EKF family	The particle filter
Belief representation	A single Gaussian (mean + covariance)	A crowd of particle samples
Distributions it can handle	Gaussian only (unimodal)	Any shape (multimodal possible)
Nonlinearity	The EKF relies on linearized approximation	Naturally supported, no linearization needed
The kidnapped	Hard to handle	Inherently solvable

Characteristic	The Kalman / EKF family	The particle filter
robot problem		
Computational cost	Low (in low dimensions)	High (many particles needed)
Representative applications	GNSS/INS, EKF-SLAM	AMCL, Gmapping

Table 11-2 The Kalman Family Compared with the Particle Filter

You now command the core estimation tools of SLAM: describing uncertainty with probability, handling it with the predict-update of Bayesian filtering, and three concrete implementations in the Kalman filter, the EKF, and the particle filter. These belong to the recursive filtering route—update once for every piece of data. But modern SLAM has another, more powerful route: rather than processing data one piece at a time, accumulate a batch and optimize it all at once. The next chapter, "Optimization and Graph-Based SLAM," introduces the route that dominates contemporary SLAM—pose graphs, bundle adjustment, and g2o, Ceres, and GTSAM.

Formulations and Procedures

1. The One-Dimensional Gaussian Distribution

The probability density function of a one-dimensional Gaussian:

$$p(x) = (1 / \sqrt{(2\pi \sigma^2)}) \cdot \exp(-(x-\mu)^2 / (2\sigma^2))$$

where μ is the mean (the center of the distribution, the best estimate) and σ^2 the variance (the magnitude of the uncertainty). About 68% of the probability falls within μ±σ, 95% within μ±2σ, and 99.7% within μ±3σ—this "68-95-99.7 rule" is the most practical intuition for reading uncertainty.

2. Bayes' Rule

The theoretical core of state estimation:

$$p(x \mid z) = p(z \mid x) \cdot p(x) \;/\; p(z)$$

Read as: posterior = (likelihood × prior) / evidence. In filtering, p(x) is the prior belief from the prediction step, p(z|x) is the observation model, and p(x|z) is the updated posterior belief. The denominator p(z) is only a normalizing constant (ensuring the probabilities sum to 1), so it is often written in proportional form:

$$p(x \mid z) \propto p(z \mid x) \cdot p(x)$$

3. The Two Steps of the Bayes Filter

Prediction step (apply the motion model and integrate over the previous state):

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t) \cdot bel(x_{t-1})\, dx_{t-1}$$

Update step (multiply by the observation model and normalize):

$$bel(x_t) = \eta \cdot p(z_t \mid x_t) \cdot \overline{bel}(x_t)$$

where $\overline{bel}$ is the predicted belief (the prior), bel the updated belief (the posterior), and η a normalizing constant. The Kalman filter, the EKF, and the particle filter are all concrete realizations of these two expressions under different representations.

4. The Core Kalman Filter Equations (Conceptual Form)

Prediction:

$$\text{mean: } \bar{x} = F \cdot \hat{x} \qquad \text{covariance: } \bar{P} = F \cdot P \cdot F^T + Q$$

Update:

$$\text{Kalman gain: } K = \bar{P} \cdot H^T \cdot (H \cdot \bar{P} \cdot H^T + R)^{-1}$$

$$\text{mean: } \hat{x} = \bar{x} + K \cdot (z - H \cdot \bar{x}) \qquad \text{covariance: } P = (I - K \cdot H) \cdot \bar{P}$$

where F is the motion model, H the observation model, Q the motion noise, and R the observation noise. Look closely at the expression for the gain K: when the observation noise R is small (the observation is accurate), K grows and the update trusts the observation more (the innovation $z - H\cdot\hat{x}^{-}$, the difference between observation and prediction, is multiplied by a large K); when R is large, K shrinks and the prediction is trusted more. This expression gives precise mathematical form to the "weighting by trustworthiness" intuition of Section 11.5.

5. A Procedure for Choosing Among the Three Filters

Figure 11-5 A Decision Procedure for Choosing a Filter

Question 1: is the system linear?

├─ Yes → the Kalman filter (KF): optimal and fastest

└─ No → go to the next question

↓

Question 2: is the nonlinearity mild, the belief unimodal, and a good initial estimate available?

├─ Yes → the extended Kalman filter (EKF): efficient, by linearized approximation

└─ No (strong nonlinearity / multimodality / the kidnapped robot problem) → go to the next question

↓

Question 3: is the state dimension low and a large number of particles affordable?

├─ Yes → the particle filter (PF / MCL): can handle any distribution

└─ No (high dimension, large scene) → consider the graph optimization of Chapter 12

11.8 Contemporary Developments in State Estimation: Filtering Has Not Left, It Has Moved

The Kalman family and the particle filter introduced in this chapter are often described in textbooks as classical methods, as though they had been superseded by the graph optimization of Chapter 12. That impression is only half right, and the distinction is

worth drawing carefully.

1. The Back End Really Has Been Taken Over by Optimization

In the back end of SLAM—the layer that maintains the whole trajectory and the whole map—optimization has indeed won out completely. The reason was given in Section 11.6: the covariance matrix of EKF-SLAM grows as the square of the number of landmarks and cannot cope beyond a few thousand; and the linearization is performed only once, so an early error cannot be revisited.

The graph optimization of Chapter 12 retains all historical poses and observations, can relinearize repeatedly, and can correct an entire trajectory at once on loop closure. Add incremental solvers such as iSAM2, which update only the affected part, and optimization is no longer at a disadvantage in speed either. ORB-SLAM3, Cartographer, and LIO-SAM today all take this route without exception.

2. But the Front End and the High-Rate Loops Remain the Domain of Filtering

Zoom in, however, to real-time estimation several hundred times a second, and filtering remains irreplaceable. Attitude estimation in a UAV flight controller, body state estimation in a quadruped, and the high-rate IMU dead reckoning of Chapter 7 are all still run by an EKF—because optimization needs a batch of data accumulated before it can solve, and a control loop cannot wait.

This yields a practical division of labor that students should internalize: high-rate, low-dimensional, real-time → filtering; low-rate, high-dimensional, globally consistent → optimization. A complete robotic system has both, layered side by side rather than chosen between. LIO-SAM in Chapter 18 and ORB-SLAM3 in Chapter 23 are both such layered architectures.

3. The Invariant EKF: Making Filtering Dependable Again

Filtering itself is advancing too. Section 11.6 noted that the fundamental weakness of the EKF is linearization error—when the estimate departs far from the truth, the Jacobian is computed at the wrong place and the filter may diverge. The recent invariant extended Kalman filter (InEKF) exploits the Lie group structure mentioned in Section 10.8 and defines the state on a matrix Lie group; under certain conditions its error dynamics are independent of the estimate, which gives it better convergence properties.

This line has been especially successful on legged robots and has become the standard approach to body state estimation on many quadrupeds and humanoids. For students it is a fine illustration: the Lie groups that looked abstract in Chapter 10 translate directly here into a more stable filter.

4. Neural Compensation: Letting a Filter Handle What It Cannot Model

The newest line hangs a learned model alongside the filter to handle the cases where the model assumptions break down. A 2026 study on legged robots is a typical example: it adds a neural compensator built around an attention mechanism on top of an invariant EKF.

Its design logic repays study. State estimation for a legged robot relies on the assumption that a planted foot does not slip, but that assumption collapses on gravel or low-friction surfaces. The study treats the degree of foot slip as an explicit conditioning variable, encodes the filter's state history and the degree of slip separately with GRU autoencoders, uses cross-attention to produce a compensation adjusted to the slip situation, and finally adds a nine-dimensional residual correction to the filter's posterior state.

The effect is considerable: position error on gravel fell by about 93% relative to the strongest baseline, on low-friction Teflon panels by about 49%, and in outdoor grass tests of about a hundred meters by about 55% to 73%, while the whole still runs in real time at 580 hertz.

Note the structure—identical to the learning-based IMU of Section 7.8 and the depth estimation models of Section 9.8: keep the geometric and probabilistic skeleton and let the neural network compensate for the parts physics cannot write down. This is already the most robust pattern for integrating AI in robotics today, and Chapter 32 discusses it again from a higher vantage.

These Four Developments in One Sentence

Filtering has not been superseded; it has merely withdrawn from taking charge of the whole map to taking charge of the immediate moment—and in that position it remains irreplaceable.

You are not studying this chapter as archaeology: writing UAV attitude estimation in Chapter 26, tuning AMCL parameters in Chapter 29, and understanding in Chapter 16 why a cleaning robot suddenly remembers where it is after getting lost all use what is in this chapter.

Case Studies

Case 11-1 The Moon Landing: The Kalman Filter's Battle of Renown

The Kalman filter was introduced by Rudolf Kálmán in 1960, and its first world-shaking application was the Apollo lunar program. A spacecraft traveling between Earth and Moon required a navigation system that fused noisy data from inertial

measurement, ground tracking, and other sources to estimate position and velocity in real time—the perfect stage for the Kalman filter's "fuse many sources by trustworthiness."

The spacecraft's computing power was extremely limited at the time (far less than a pocket calculator today), but the Kalman filter needs to track only two quantities, the mean and the covariance, and computes recursively (with no need to store all past data), so it could run in real time on such straitened hardware. One may fairly say that without the Kalman filter, Apollo's navigation would have been nearly impossible.

This case echoes Apollo's gimbal lock in Chapter 10—one and the same lunar mission drove the development of both rotation representation and state estimation. It also offers a lesson: these seemingly abstract probabilistic tools are the practical bedrock on which humanity walked on the Moon. More than sixty years later, the positioning in your phone and the navigation of your cleaning robot still use the same mathematics.

Case 11-2 The Cleaning Robot's "Particle Cloud": Visualizing AMCL

Many cleaning robot apps that display a map will, in a debug or advanced mode, show a vivid scene: at power-on, a great many small dots (particles) representing possible positions are scattered over the map; as the robot moves a few steps and scans a few times, those dots converge rapidly from "scattered over the whole map" into "a tight little cluster"—the robot has worked out where it is.

This is exactly a live visualization of the AMCL of Section 11.7. Scattered particles mean "I am not yet sure where I am" and a converged cluster means "now I am sure." Carry the robot into another room and the cluster will first become confused

(inconsistent with the observations) and then regather at the correct new position—the solving of the kidnapped robot problem unfolding before your eyes.

The teaching value of this case is that it turns the chapter's most abstract idea, a probability distribution, into a visible cloud of particles. In university classrooms, instructors often have students watch the particles evolve in AMCL within RViz, and alongside the theory of this chapter the abstract "contraction of belief" becomes concrete at once. It is another example of this book's emphasis on understanding the principle first and then observing it by hand.

Case 11-3 Sensor Fusion in a Self-Driving Car: The Gain Balancing Trust on a Real Road

A self-driving car carries GPS, an IMU, wheel encoders, LiDAR, and cameras at once, each with its own account of where the car is and each dependable at different moments. The Kalman family (usually an EKF or a variant) is the mainstay of fusing this information.

The Kalman gain of this chapter is embodied here very concretely: when the vehicle enters a tunnel and GPS loses lock (the observation noise R soars), the filter automatically lowers its trust in GPS and leans on the dead reckoning of the IMU and the wheel encoders (the integrated navigation of Chapter 5); after leaving the tunnel and GPS recovers, the filter gradually shifts trust back. This dynamic adjustment of trust by situation needs no hard-coded rules from an engineer; it follows naturally from the mathematics of the covariance and the gain.

This is precisely the fundamental advantage of the probabilistic approach over hard-coded rules: it lets the system weigh each information source by its present

trustworthiness and adapt gracefully to any situation. The idea runs through the whole book—from the indoor positioning fusion of Chapter 4 and the GNSS/INS of Chapter 5 to the attitude filtering of Chapter 7 and the unified mathematical statement of this chapter, you should be able to feel that Parts II and III are really treating different faces of one thing.

Hands-On Activities

Exercise 11-1 Building a One-Dimensional Kalman Filter and Particle Filter by Hand

Purpose: using the simplest one-dimensional example, to implement a Kalman filter and a particle filter by hand and see with your own eyes how uncertainty widens and contracts and how a particle cloud converges. This exercise is mainly in Python and is the key step to understanding all the theory of this chapter.

Part One: A One-Dimensional Kalman Filter (everyone)

1. Set up the scenario: a robot on a one-dimensional track, true position unknown. It has a drifting velocity sensor (the motion model, with large noise) and a position sensor that reports occasionally (the observation model, with moderate noise). Generate a stretch of noisy simulated data in code.
2. Implement the prediction step of the Kalman filter: at each step update the mean position with the velocity and enlarge the variance. Plot the estimated position and the uncertainty band (mean ± standard deviation) against time, and observe how the band keeps widening with prediction alone.
3. Add the update step: whenever the position sensor reports, correct the mean with the observation and shrink the variance. Observe how the uncertainty band contracts at each observation.

4. Plot the estimated trajectories of pure prediction (dead reckoning) and the complete Kalman filter together, compare them with the truth, and quantify how much error the Kalman filter removed.
5. Experiment with the Kalman gain: set the observation noise R very large and then very small, and observe how the gain changes and how the estimated trajectory leans more toward prediction or toward observation.

Part Two: A One-Dimensional Particle Filter (everyone, or undergraduate)

6. Scatter 500 particles (uniformly distributed) along the one-dimensional track, representing "not knowing where the robot is at first."
7. Implement the three steps: at each step move all particles by the motion model (with noise added), weight each particle by the observation, and resample by weight.
8. Plot a histogram of the particle distribution every few steps and observe how the cloud converges from "spread over the whole track" to "near the true position."
9. Kidnapping test: move the true position elsewhere abruptly partway through and observe how the cloud first becomes confused and then regathers at the new position.

Part Three: A Multimodal Scenario (undergraduate)

10. Design a symmetric corridor scenario in which the observations at two locations are identical (a symmetric environment). Handle it with a particle filter and observe how the cloud maintains two peaks—verifying first-hand the claim of Section 11.7 that a particle filter can handle multimodality and a Kalman filter cannot.

11. Consider: what would happen if a Kalman filter handled this symmetric scenario? (Hint: a single Gaussian would place the mean midway between the two peaks—a position the robot cannot possibly occupy.)

Discussion Questions

1. In Part One, how much error did the complete Kalman filter remove relative to pure dead reckoning? Where does that improvement come from?
2. When you set the observation noise R very small, what happened to the estimated trajectory? Does this match the account of the Kalman gain in Section 11.5?
3. In the particle filter kidnapping test, how long did the cloud take to regather? What would happen with only 50 particles rather than 500? What trade-off regarding the number of particles does this reveal?
4. (Undergraduate) In the symmetric corridor scenario the particle filter maintained two peaks while a Kalman filter would fail. What practical lesson does this offer about choosing a filter?

Extended Challenge (Undergraduate)

Extend the one-dimensional Kalman filter to two dimensions (x, y), track the covariance matrix, and plot the uncertainty ellipse at every step with matplotlib; observe how the ellipse stretches along the direction of travel as the robot moves in a straight line and how it contracts after an observation. This two-dimensional visualization corresponds directly to Section 11.2 and builds the intuition for Gmapping in Chapter 14 and the AMCL exercises of Chapter 29.

Further Reading

1. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The bible of probabilistic robotics and the authoritative source for every topic in this chapter; Chapters 2 through 8 are especially relevant.)
2. Cyrill Stachniss, the Bayes Filter, Kalman Filter, and Particle Filter video series from his SLAM course (YouTube). (Explains all three filters intuitively with animation; the first choice for self-study.)
3. R. Faragher, "Understanding the Basis of the Kalman Filter via a Simple and Intuitive Derivation," IEEE Signal Processing Magazine, 2012. (The friendliest intuitive derivation of the Kalman filter.)
4. R. Labbe, "Kalman and Bayesian Filters in Python" (a free e-book on GitHub). (With complete runnable code; the best companion for implementing this chapter.)
5. D. Fox et al., "Monte Carlo Localization for Mobile Robots," ICRA, 1999. (The original paper on Monte Carlo localization.)
6. The filtering units of the CMU 16-833 Robot Localization and Mapping course (introduced in Chapter 2). (Graduate-level material on EKF-SLAM and the particle filter.)
7. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, the first part of Lecture 6 (nonlinear optimization). (The bridge from the filtering of this chapter to the graph optimization of Chapter 12.)

From Filtering to Optimization: Where This Chapter Stands

This chapter has written the SLAM problem as a probabilistic one and given three ways of solving it. But all three share one character: they are recursive—they keep only the current belief, update it whenever new data arrives, and then discard the old data.

The advantage is fixed memory and steady computation, very well suited to a real-time system. But the price is equally clear: once an estimate at some step goes wrong, the error is pressed into the current belief and can never be revisited. When the robot comes all the way around to its starting point and finds the map does not line up, a filter can only correct slowly from that moment on; it cannot go back and straighten the whole stretch of trajectory in between.

What Chapter 12 introduces is the opposite approach: throw nothing away. Treat the pose at every instant as a variable to be solved for, treat every observation as a constraint, and put them all into one enormous optimization problem to be solved at once. The moment loop closure detects "I am back at the start," that new constraint pulls the entire trajectory back to the right place—something filtering can never do.

Interestingly, the two have the same mathematical root: the maximum a posteriori estimate that optimization seeks and the peak of the posterior distribution that filtering pursues are, under Gaussian assumptions, the same thing. Chapter 12 explains this equivalence. So nothing learned in this chapter goes to waste—it has merely changed its manner of computation.

Summary

1. SLAM must handle uncertainty probabilistically: every sensor gives a noisy guess, and the mature approach is to quantify the uncertainty and then strike the best balance. The key idea is that what is estimated is a probability distribution (a best guess plus a trustworthiness), not a single value.
2. A Gaussian distribution is described completely by two parameters, the mean (the best estimate) and the variance (the uncertainty); in several dimensions the

covariance matrix corresponds to an uncertainty ellipse. The Gaussian is the first choice in SLAM because two parameters suffice, it is closed under the relevant operations, and the central limit theorem supports it—but it cannot describe a multimodal distribution.

3. The motion model p(x_t|x_{t−1},u_t) describes where motion leads and enlarges the uncertainty; the observation model p(z_t|x_t) describes what would be seen from here and contracts it. The two are the inputs to every filter.
4. The Bayes filter is the common skeleton of every filter, recursing through two steps—prediction (apply the motion model, the belief widens) and update (apply the observation model, the belief narrows)—with the update grounded in Bayes' rule: posterior ∝ likelihood × prior.
5. The Kalman filter is the optimal solution under linear Gaussian assumptions and need track only the mean and the covariance; the Kalman gain balances prediction against observation by their trustworthiness—the mathematical core of why multi-sensor fusion beats any single sensor.
6. The EKF handles nonlinearity by local linearization with a Jacobian and is the most widely used estimator in robotics; EKF-SLAM places poses and landmarks in one state and estimates them jointly, embodying the coupling of localization and mapping, but its covariance grows as the square of the number of landmarks and its linearization error accumulates—which gave rise to the graph optimization route.
7. The particle filter approximates an arbitrary distribution with a crowd of samples through three steps—prediction, weighting, resampling—and can handle multimodality and nonlinearity and inherently solve the kidnapped

robot problem; Monte Carlo localization (AMCL) is its localization application and the Rao-Blackwellized version (Gmapping) its SLAM application.

The filters of this chapter are all recursive—update once per piece of data and then discard it. That is efficient, but it also means that once past information has been compressed into the current belief it can never be revisited. The next chapter, "Optimization and Graph-Based SLAM," introduces the other route: retain a batch of poses and observations, build them into a pose graph, and optimize all at once for the most consistent solution. This route is more accurate and handles loop closure better, and it is the mathematical foundation of contemporary mainstream systems such as ORB-SLAM and Cartographer.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 12

Optimization and Graph-Based SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 11 wrote SLAM as a probabilistic problem and gave three filtering solutions. But filtering has one common character: it keeps only the current belief and discards the old data at every update. That is economical in real time but costly in correctness—once an error has been pressed into the belief, it can never be revisited.

What this chapter introduces is the opposite approach: throw nothing away. Treat the pose at every instant as a variable to be solved for, treat every observation as a constraint, and put them all into one optimization problem to be solved at once. When the robot comes around to its starting point and finds the map does not line up, the new loop closure constraint can straighten the entire trajectory—something filtering can never do.

This is the mainstream architecture of modern SLAM. The 2D SLAM of Chapters 14 and 15, the LiDAR-inertial odometry of Chapter 18, and the ORB-SLAM family of Chapters 22 and 23 all have the graph optimization of this chapter as their back end. Of the three chapters in Part III, this is the one that connects most directly to implementation.

Figure 12-0 How Chapter 12 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 10 The pose T → becomes a node of the graph here

Chapter 11 Probability and information → becomes the information matrix Ω on an edge here

Chapter 11's filtering "discards history" → this chapter "retains all history"

[This chapter is responsible for]

The front-end/back-end division → the pose graph → least squares → bundle

adjustment

Sparsity: why even tens of thousands of nodes remain solvable

g2o, Ceres, and GTSAM: the tools you will actually use

[Where it leads]

→ Chapter 15 Cartographer's branch-and-bound loop closure + a pose graph back end

→ Chapter 17 An ICP registration result = one edge of a pose graph

→ Chapter 18 LIO-SAM: a factor graph of LiDAR, IMU, and GNSS

→ Chapters 22 and 23 ORB-SLAM's local BA and global BA

→ Section 7.5 What IMU preintegration produces is one edge of this chapter's graph

Learning Objectives

The filters of the previous chapter are recursive—update once per piece of data and then discard it. This chapter introduces the other route: retain a batch of poses and observations, build them into a graph, and optimize all at once for the most consistent solution. This graph optimization route is more accurate and handles loop closure better; it is the mathematical foundation of contemporary mainstream systems such as ORB-SLAM and Cartographer, and the conclusion of Part III. After completing this chapter, you should be able to:

1. Describe the division of labor between the SLAM front end and back end, and understand that graph optimization belongs to the back end.
2. Describe what the nodes and edges of a pose graph represent, and read how such a graph describes a SLAM problem.
3. Explain why SLAM can be cast as a least-squares problem.
4. Explain the concept of bundle adjustment and its role in visual SLAM.

5. Know the place and purpose of the three major optimization libraries g2o, Ceres, and GTSAM.
6. Explain why modern SLAM mostly adopts graph optimization rather than the filters of the previous chapter.

Core Concepts

Chinese Term	English Term	Explanation
前端	Front-End	Turns raw sensor data into geometric constraints (features, matches, odometry).
後端	Back-End	Finds the most consistent poses and map under all the constraints.
位姿圖	Pose Graph	A graph structure with poses as nodes and constraints as edges.
節點	Node / Vertex	A quantity to be estimated, such as a robot pose or a landmark position.
邊	Edge / Constraint	A relative measurement constraint between two nodes.
最小平方	Least Squares	The method of finding the solution minimizing the sum of squared errors.
殘差	Residual	The difference between a measured value and a predicted value.
光束法平差	Bundle Adjustment, BA	Visual optimization of camera poses and three-dimensional points together.
重投影誤差	Reprojection Error	The gap between a three-dimensional point's projection into an image and the pixel actually observed.
資訊矩陣	Information Matrix	The inverse of the covariance, representing a constraint's trustworthiness weight.

Chinese Term	English Term	Explanation
稀疏性	Sparsity	Each constraint involves only a few variables, which lets a large problem be solved efficiently.
圖優化函式庫	g2o / Ceres / GTSAM	Open-source tools for solving graph optimization problems.

12.1 The Division of Labor Between Front End and Back End

Modern SLAM systems generally adopt a two-stage front-end-plus-back-end architecture (the common skeleton of the university courses of Chapter 2 and the core of Figure 1-3 in Chapter 1). Understanding this division is the first step to reading this chapter.

The Front End: Turning Sensor Data into Geometric Constraints

The front end faces the sensors directly, and its job is to turn raw data into geometric constraints—relations such as "between position A and position B the robot moved so far" or "a certain landmark was seen at position A."

- For LiDAR SLAM: the front end performs scan matching (Chapter 13) to compute how far this frame moved relative to the previous one.
- For visual SLAM: the front end performs feature extraction and matching (Chapters 9 and 20) to compute the camera motion and the landmarks seen.
- The front end is also responsible for loop closure detection: recognizing "I have been to this place" and producing a crucial loop closure constraint.

The Back End: Finding the Most Consistent Solution Under All Constraints

The back end does not touch the sensors directly; it receives the heap of geometric constraints the front end produces, and its task is to find a set of poses and a map that satisfy all the constraints as nearly as possible at once.

Here is the difficulty: these constraints often contradict one another. Odometry says "I moved 10 meters from A to B," while loop closure detection says "B is in fact right beside A"—the two constraints are at odds. The back end's job is to find the best compromise among such contradictory constraints so that the total error is least. That

is the subject of this chapter.

Two Routes for the Back End

Historically there have been two routes to finding that best compromise:

- The filtering route (Chapter 11): process recursively, one piece at a time, as represented by the EKF and the particle filter. Its advantages are real-time operation and fixed memory; its drawback is that past information, once compressed, cannot be revisited.
- The optimization route (this chapter): retain a batch of constraints and solve the whole problem at once, as represented by graph optimization. Its advantages are high accuracy, repeated correction, and elegant handling of loop closure; its price is greater computation.

Over the past decade and more, the optimization route has become mainstream thanks to its accuracy and its handling of loop closure. This chapter sets out the mathematics and tools of that route.

12.2 The Concept of the Pose Graph

The central data structure of graph-based SLAM is a pose graph. It turns the abstract SLAM problem into an intuitive graph—the very graph of nodes and edges studied in a data structures course.

Nodes and Edges

- Node (vertex): represents a quantity to be estimated. In the simplest pose graph, each node is the robot's pose at some instant (the x, y, θ of Chapter 10, or a 6-DoF pose). In versions that include landmarks, landmark positions are nodes as well. These are the unknowns we want to solve for.

- Edge (constraint): represents a relative measurement constraint between two nodes. It records what relation some sensor says should hold between those two nodes.

Two Crucial Kinds of Edge

A pose graph has two kinds of edge of different origin but equal importance:

- Odometry edges (sequential edges): connect two poses adjacent in time, coming from odometry or scan matching—"I moved this much from frame t to frame t+1." These edges form a chain that strings the poses together.
- Loop closure edges: connect two poses not adjacent in time but reunited in space, coming from loop closure detection—"where I am now is the same place as frame 5 long ago." Such an edge forms a loop in the graph and is the key to eliminating accumulated drift.

Why Loop Closure Edges Matter So Much

Imagine a robot going once around a building and returning to its starting point. The odometry edges form a chain accumulating error all the way, so that after the circuit a noticeable gap appears between the endpoint and the start (the drift of Chapter 1). Loop closure detection then produces an edge declaring "the endpoint is the start"—and this edge contradicts the entire odometry chain.

The magic of back-end optimization is to spread that contradiction evenly over the whole trajectory: it nudges every pose along the way so that the endpoint returns to the start while disturbing the odometry constraints as little as possible. The result is that the whole trajectory is straightened and the accumulated drift is removed at a stroke. This is why loop closure is so crucial to SLAM, and a great advantage of graph optimization over filtering: it can go back and correct every historical pose.

Figure 12-1 The Structure of a Pose Graph

Nodes: ①—②—③—④—⑤—⑥—⑦—⑧ (each one a pose)

└——odometry edges (adjacent, like a chain)——┘

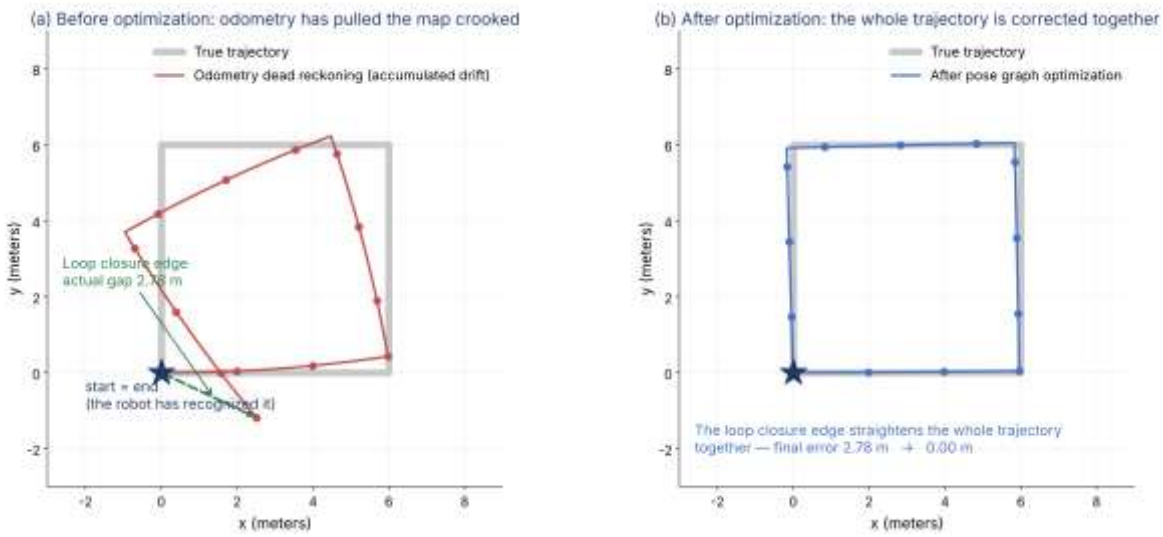
① ←————loop closure edge————→ ⑧

(frame ⑧ is detected to have returned to the position of frame ①)

Before optimization: the odometry chain has accumulated drift and ⑧ does not line up with ①

After optimization: the loop closure edge spreads the contradiction over ①–⑧ and the whole trajectory is straightened

Figure 12-2 The Power of Loop Closure: How One Edge Straightens a Whole Map



This figure is actually computed, not a sketch. A robot is simulated walking once around a 6-meter square, with odometry carrying a systematic angular bias of about 0.013

radian per step—a very typical source of error in real wheeled robots, arising for instance from a slight difference in the diameters of the left and right tires.

The left panel is the result of odometry alone. Note its shape: each leg is still fairly straight and the corners are close to 90 degrees, so it looks entirely plausible—yet the whole has rotated askew. On returning to the start after a full circuit, the dead-reckoned position is 2.56 meters from the true starting point. This is what the integration drift of Chapter 7 and the "prediction can only raise uncertainty" of Chapter 11 look like in a plane.

The crucial step occurs the moment the robot recognizes "I am back at the start." That recognition is performed by the front end—scan matching in Chapters 14 and 15, the visual bag of words in Chapters 20 and 22. The front end's conclusion is written as a loop closure edge: the relative pose between node N and node 0 should be zero.

The right panel is the result of optimizing with that edge added. The whole trajectory is straightened together, the final error falls from 2.56 meters to 0.00 meters, and the mean error along the route falls from 1.18 meters to 0.04 meters. Note the word together—it is not only the last node that is corrected; every node in between shares in the correction. This is the most fundamental advantage of graph optimization over filtering: it can go back and rewrite history.

Why Filtering Cannot Do This

At step k a filter has already compressed everything before step k into one mean and one covariance; the positions of the intermediate nodes no longer exist in the state and so cannot be corrected.

Graph optimization retains all N nodes as variables, so the new information

from a loop closure edge can propagate back along the chain of odometry edges and move every node a little.

The price is more variables and storing all of history. That is the trade-off of Section 12.7, and it is why practice often layers the two together: filtering in the front end to maintain high rate, graph optimization in the back end to maintain global consistency.

12.3 Nodes, Edges, and Constraints

This section looks more closely at the constraints that edges represent and at how to quantify the degree to which a constraint is satisfied—the key to turning a pose graph into a solvable problem.

The Residual: The Difference Between Measurement and Prediction

Every edge carries a measured value (what relation the sensor says should hold between the two nodes). From the current estimates of the two nodes' poses we can also compute a predicted value (what relation actually holds between them under the current estimate). Their difference is called the residual:

$$\textit{residual} = \textit{measured value} - \textit{predicted value}$$

A zero residual means this edge's constraint is perfectly satisfied; the larger the residual, the more the current pose estimate violates that constraint. The back end's goal is to adjust all the nodes so that the residuals of all the edges are as small as possible.

Not All Constraints Are Equally Trustworthy: The Information Matrix

A key detail here: different edges differ in trustworthiness. An edge from high-

accuracy LiDAR scan matching is far more trustworthy than one from slipping odometry. The optimization should respect the trustworthy edges more.

That trustworthiness is described by an information matrix—which is exactly the inverse of the covariance matrix of Chapter 11. A large covariance (uncertain) corresponds to a small information matrix (low weight); a small covariance (certain) to a large information matrix (high weight). Trustworthy constraints thus speak louder in the optimization. This joins graph optimization perfectly to the probabilistic framework of Chapter 11: the two are different solutions to the same probabilistic problem.

The Uniformity of Constraints

The beauty of the pose graph is that it unifies constraints of every origin—odometry, scan matching, loop closure, even the UWB ranging of Chapter 4, the GNSS of Chapter 5, and the IMU preintegration of Chapter 7—into edges between nodes. Whatever sensor a constraint comes from, in the graph it is an edge with a residual and an information matrix. This uniformity is why graph optimization fuses multiple sensors so elegantly, and it echoes the tight coupling idea recurring in earlier chapters: put all the raw constraints into one optimization problem and solve them together.

12.4 Least Squares

Now let us gather the foregoing into a definite mathematical problem. Since the residuals of all edges are to be made as small as possible, the natural approach is to find a set of node values minimizing the sum of the squares of all residuals. This is least squares.

Why the Sum of Squares

Why the sum of squares rather than the sum of absolute values of the residuals?

For two reasons:

- A mathematical reason: the square function is differentiable everywhere and has a pleasant form, which makes it convenient to find a minimum by calculus (setting the derivative to zero). The absolute value is not differentiable at zero and is awkward to handle.
- A probabilistic reason: when the noise is Gaussian (Chapter 11), minimizing the sum of squared residuals is mathematically equivalent to maximizing the posterior probability (finding the most probable solution). In other words, least squares is not chosen arbitrarily; it is the optimal estimate under the Gaussian assumption. This shows once again that graph optimization and the probabilistic framework of Chapter 11 are the same thing.

With the information matrix added as a weight, the more precise statement is "minimize the weighted sum of squared residuals"—trustworthy edges carry greater weight and influence the sum more.

How It Is Solved: Successive Approximation

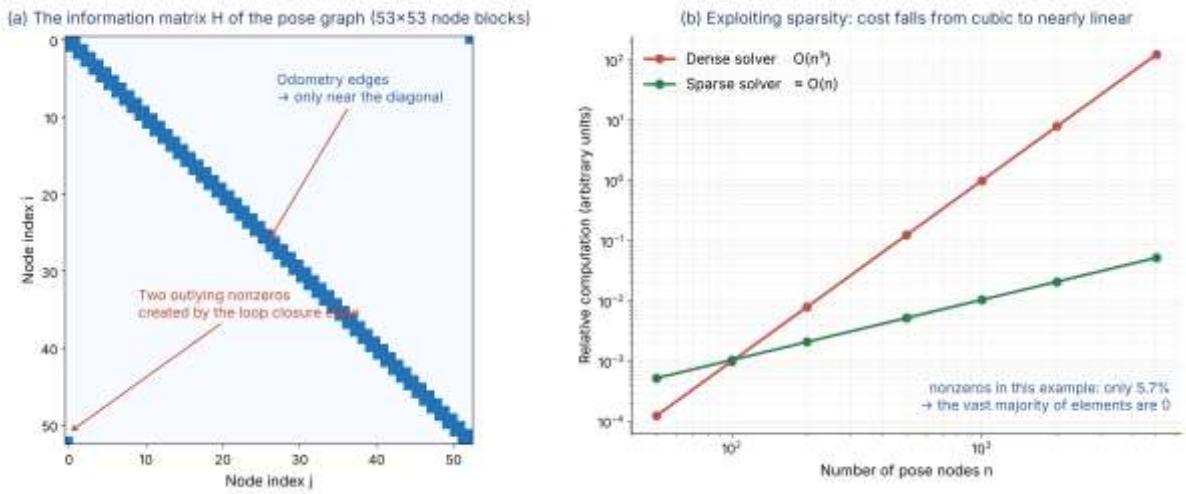
This least-squares problem is usually nonlinear (because the residuals contain cosθ, sinθ, and the like, from Chapter 10) and cannot be solved in one step. In practice it is approached iteratively:

1. Begin from an initial guess (usually the rough trajectory strung together from odometry).
2. Linearize the problem near the current estimate (computing a Jacobian, the same trick the EKF used in Chapter 11).
3. Solve a linear system to obtain a step correcting in the direction of smaller error.

4. Update the estimate and return to step 2, repeating until the residual no longer falls appreciably (convergence).

Common iterative methods are Gauss-Newton and Levenberg-Marquardt; the latter is more robust and is the default choice in many SLAM systems. You need not compute these by hand—that is precisely what the libraries g2o, Ceres, and GTSAM do for us (Section 12.6).

Figure 12-3 Sparsity: The Secret of Solving Tens of Thousands of Nodes in Real Time



The left panel plots the structure of the information matrix of the pose graph of Figure 12-2, with dark cells marking nonzeros. Nearly all nonzero elements are crowded near the diagonal—because each odometry edge connects only two adjacent nodes and involves no others. The only exceptions are the two isolated blocks at upper right and lower left, created by the loop closure edge that joins the first node to the last.

In this small example of 53 nodes the nonzero fraction is already only 5.7%; raise the node count into the thousands and it falls below one part in a thousand. In other words, this matrix is almost entirely zero.

The right panel shows why that matters. A general dense solver costs $O(n^3)$ operations,

so a thousand nodes require on the order of a billion operations and ten thousand nodes a trillion—quite impossible in real time. But a sparse solver (sparse Cholesky factorization, the conjugate gradient method, and the like) handles only the nonzeros and costs close to O(n), so even tens of thousands of nodes are solved within tens of milliseconds.

This is not a detail but the engineering basis on which modern SLAM exists at all. More than half of the value of the three libraries g2o, Ceres, and GTSAM of Section 12.6 lies in exploiting sparsity to the utmost. When you compile ORB-SLAM2 in Chapter 22 and install g2o, recall this figure—a matrix of this shape is what it handles.

Sparsity: The Secret That Makes Large Problems Solvable

A pose graph of a large scene may have tens of thousands of nodes, corresponding to tens of thousands of unknowns. Solving a linear system that large directly would be computationally infeasible. Fortunately the SLAM problem has a saving property: sparsity.

Sparsity means that each edge involves only two nodes rather than all of them. Hence in the large matrix after linearization the great majority of elements are zero. Exploiting that structure, specialized sparse matrix algorithms can solve extremely quickly—which is precisely what EKF-SLAM (Chapter 11) cannot do and graph optimization can: the EKF's covariance matrix becomes dense (every landmark becomes correlated with every other), while graph optimization stays sparse. This is the mathematical root of graph optimization's ability to handle large scenes.

12.5 Bundle Adjustment

The pose graph above handles constraints between poses. In visual SLAM there is

a finer optimization problem that handles camera poses and three-dimensional map points at once—bundle adjustment (BA). It is the source of accuracy in visual SLAM and the core of ORB-SLAM in Chapters 22 and 23.

Where the Name Comes From

"Bundle" refers to the many rays emanating from each three-dimensional point toward each of the cameras that see it. BA's task is to adjust all camera poses and all three-dimensional point positions so that these bundles of rays align as nearly as possible—that is, so that each three-dimensional point projects into an image as close as possible to the pixel actually observed. That difference between projection and observation is called the reprojection error.

The Reprojection Error: The Residual of Visual SLAM

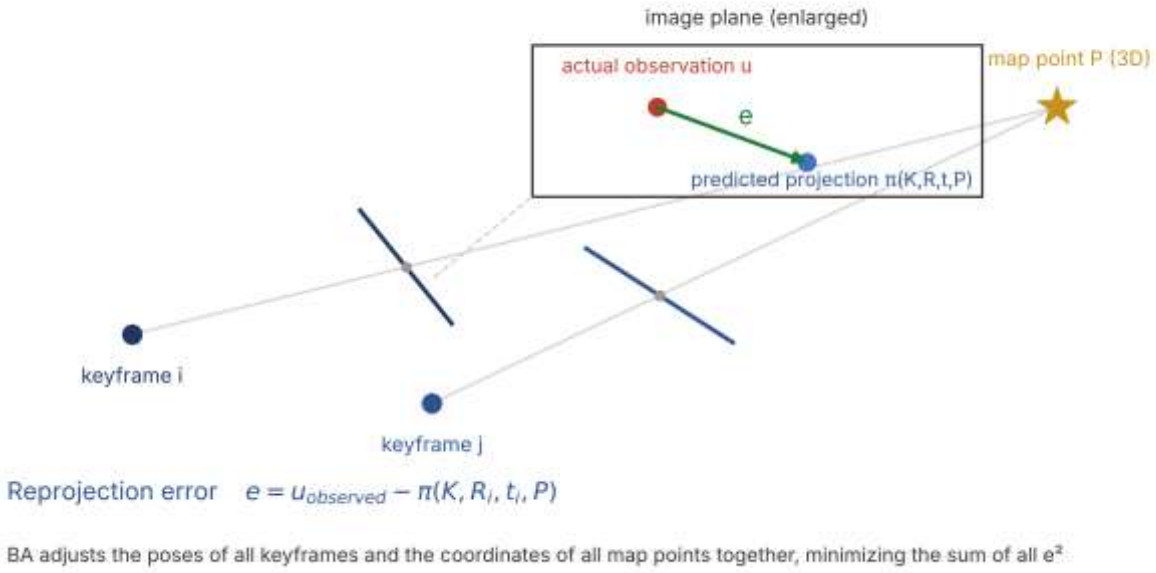
The reprojection error is BA's central residual, and it puts the camera projection model of Chapter 9 to work:

5. Take a three-dimensional map point and project it into the image with the currently estimated camera pose and the intrinsics K (Chapter 9), obtaining a predicted pixel position.
6. That map point also has an actually observed pixel position in that image (obtained by front-end feature extraction).
7. The difference between the two (in pixels) is the reprojection error.

BA adjusts all camera poses and all three-dimensional points at once to minimize the sum of the squares of all reprojection errors. This is an enormous least-squares problem—a scene may have hundreds of camera poses and tens of thousands of three-dimensional points—but it too is solved efficiently through sparsity (each point

is seen by only a few cameras).

Figure 12-4 The Reprojection Error: The e That Bundle Adjustment Minimizes



The difference between bundle adjustment and a pose graph lies in the definition of the residual. A pose graph's residual is a relative relation between two poses; BA's residual is the green arrow in the figure: project the three-dimensional map point P into the image with the currently estimated camera pose and intrinsics, and take the gap between that projection and the feature point actually detected.

This definition has two important properties. First, it is measured directly in pixels—the error is "so many pixels off," which is very intuitive and convenient for setting thresholds (the outlier test commonly used in ORB-SLAM is a reprojection error exceeding some number of pixels). Second, it involves both camera pose and map point coordinates, so BA adjusts the two together—which is where the "bundle" in its name comes from: all the bundles of rays are adjusted together.

Read this figure alongside Figure 9-1 of Chapter 9. There the function π computed a pixel position from a known camera pose and map point; here the direction is

reversed—given the pixel position, adjust the camera pose and the map point so that the computed pixel position comes as close as possible to the actual observation. That is what the whole back end of visual SLAM does.

BA has far more variables than a pose graph—each map point is three variables, and a medium-sized scene easily has tens of thousands of points. Fortunately its sparse structure is even nicer: map points have no direct edges among themselves and are connected only through cameras. Eliminating with the Schur complement by exploiting that structure lets one solve for the camera poses first and back-substitute the map points, which is the key trick that lets BA run in real time; Chapter 22 returns to it.

The Relationship Between BA and the Pose Graph

- Bundle adjustment: optimizes camera poses and three-dimensional points together with the highest accuracy, but has many variables and heavy computation. Suited to local use (the Local Mapping thread of Chapter 22) or to critical moments.
- Pose graph: optimizes poses only (marginalizing the three-dimensional points away), with few variables and high speed. Suited to large-scale loop closure correction (the Loop Closing of Chapter 22).

Modern visual SLAM combines the two ingeniously: local BA maintains high accuracy in ordinary operation, while a pose graph straightens a large stretch of trajectory quickly on loop closure. This "local BA + global pose graph" architecture is the classic design of the ORB-SLAM family, unfolded in full in Chapters 22 and 23. The theory of BA is also exactly what the MIT VNAV and CMU 16-833 courses of Chapter 2 emphasize—you should now be able to see why those leading courses devote so much space to optimization.

12.6 g2o, Ceres, and GTSAM

The mathematics above looks complicated, but the good news is that you hardly need to implement the iterative solving and sparse matrix operations yourself. Three mature open-source libraries have done the heavy lifting. Knowing where each stands is essential knowledge for entering SLAM implementation.

The Three Major Libraries

- g2o (general graph optimization): a C++ library designed specifically for graph optimization, developed by the SLAM community. Its usage is very close to the concepts of this chapter—you define nodes (poses, landmarks) and edges (constraints) and it solves for you. ORB-SLAM2 uses g2o. For a learner, g2o's architecture corresponds most closely to this chapter's pose graph concepts and is a good starting point for understanding graph optimization.
- Ceres Solver: a general nonlinear least-squares library developed by Google, not limited to SLAM. It is powerful, well documented, and convenient in its automatic differentiation (no need to compute Jacobians by hand), and it is widely used for optimization problems of every kind. Cartographer (Chapter 15) uses Ceres.
- GTSAM (Georgia Tech Smoothing and Mapping): developed at the Georgia Institute of Technology (Professor Kaess of CMU 16-833, mentioned in Chapter 2, is a core contributor), built around the concept of a factor graph and providing incremental solving algorithms such as iSAM2—able to update efficiently as new data arrives rather than recomputing from scratch each time.

The Factor Graph: An Equivalent Point of View

The factor graph used by GTSAM is a more general formulation of the pose graph.

In a factor graph the variables to be estimated are one kind of node and each constraint (called a factor) is another kind, connected to the variables it involves. It is mathematically equivalent to a pose graph, but its formulation is more flexible and accommodates sensor constraints of every sort naturally. As Chapter 2 noted, the factor graph is the central tool of the CMU 16-833 course—another sign that the content of this chapter aligns fully with leading international courses.

Library	Developer	Central Concept	Representative User
g2o	The SLAM community	General graph optimization	ORB-SLAM2
Ceres	Google	General nonlinear least squares	Cartographer
GTSAM	Georgia Tech	Factor graphs + incremental solving	LIO-SAM and others

Table 12-1 The Three Major Graph Optimization Libraries Compared

The advice for readers of this book is: at the high school and introductory undergraduate stage, understanding the pose graph–residual–least squares concepts suffices, with no need to go deep into library details; at the undergraduate project and research stage, start with g2o (closest in concept to this chapter) and then choose Ceres or GTSAM as the project requires. The exercises of Chapters 22 and 30 put these tools to actual use.

12.7 Why Modern SLAM Commonly Uses Graph Optimization

To close the chapter, let us answer a question running through Chapters 11 and 12:

since the filters of Chapter 11 can already do SLAM, why have nearly all modern mainstream systems moved to graph optimization? This contrast is the key to understanding where contemporary SLAM is going.

Four Advantages of Graph Optimization

8. It can go back and correct history: a filter compresses the past into the current belief and cannot revisit it; graph optimization retains the whole graph and can correct all historical poses at once on loop closure, flattening drift elegantly. This is the largest source of the difference in accuracy.
9. Repeated relinearization gives higher accuracy: the EKF linearizes each measurement only once (and cannot redo it); graph optimization relinearizes repeatedly through its iterations, approaching the true solution step by step and greatly reducing the linearization error noted in Chapter 11.
10. Exploiting sparsity, it handles large scenes: graph optimization's matrices stay sparse and, with sparse algorithms, handle tens of thousands of nodes; EKF-SLAM's covariance matrix becomes dense and bogs down as it grows with the square of the number of landmarks.
11. It fuses multiple sensors uniformly: every constraint is an edge of the graph, so odometry, vision, LiDAR, IMU, and GNSS can be tightly coupled naturally within one problem (the tight coupling idea of Chapter 7).

The Price of Graph Optimization

Graph optimization is not without cost. It must retain more historical data (greater memory), a single batch solve is computationally heavier, and implementation is more complex. In the early days, limited computing power made filtering the mainstream; it was the growth of computing power and the maturing of sparse solvers that made

graph optimization possible and eventually victorious. This is also a reminder: the mainstream of a technology shifts as hardware and algorithms evolve, and understanding why matters more than memorizing what is used now.

Item of Comparison	Filtering (Chapter 11)	Graph optimization (this chapter)
Manner of processing	Recursive, updating one piece at a time	Batch, optimizing all at once
Correcting history	Cannot go back	Can correct all of history
Linearization	Once per measurement	Repeatedly through the iterations
Handling loop closure	Weaker	Its strength
Large scenes	The EKF is limited (dense)	Exploits sparsity, scales well
Computation and memory	Fixed and light	Heavier
Representative systems	AMCL, early EKF-SLAM	ORB-SLAM, Cartographer

Table 12-2 An Overall Comparison of Filtering and Graph Optimization

Part III, "The Mathematical Foundations of SLAM," is now complete. You have assembled the full mathematical toolbox of SLAM: the coordinate transformations of Chapter 10 let us convert precisely between frames, the probability and filtering of Chapter 11 let us handle uncertainty, and the optimization and graph optimization of this chapter let us find the most consistent global solution. These three chapters are abstract, but they are the common bedrock of every concrete system that follows. From the next part onward we put this mathematics to use—Part IV begins with the most

approachable subject, 2D SLAM and the cleaning robot, and shows how Gmapping builds maps with the particle filter of Chapter 11 and how Cartographer closes loops with the graph optimization of this chapter. Theory and practice are about to meet on a real robot.

Formulations and Procedures

1. The Least-Squares Objective of SLAM

At the core of graph-based SLAM is finding a set of states X (all poses and landmarks) minimizing the weighted sum of squared residuals:

$$X^* = \operatorname{argmin} \Sigma_k \; e_k(X)^T \cdot \Omega_k \cdot e_k(X)$$

where e_k is the residual of the k-th edge (the difference between measurement and prediction), Ω_k the information matrix of that edge (its trustworthiness weight, the inverse of the covariance), and X* the optimal solution. This expression condenses all of SLAM into one sentence: find a set of poses and a map minimizing the weighted contradiction among all constraints. Maximizing probability as in Chapter 11 is, under Gaussian assumptions, exactly equivalent to this least-squares problem.

2. The General Form of the Residual

In a pose graph, an edge connecting nodes i and j has residual:

$$e_{ij} = \text{measured relative pose } (z_{ij}) \ominus \text{predicted relative pose } (x_i, x_j)$$

The symbol $\ominus$ denotes "subtraction" in pose space (Chapter 10)—since a pose contains a rotation, this subtraction is not ordinary subtraction but must be performed on the rotation group (the manifold operations of MIT VNAV in Chapter 2). A zero residual means the constraint is perfectly satisfied.

3. The Reprojection Error (Bundle Adjustment)

In visual SLAM, the reprojection error of a three-dimensional point P in the image of the i-th camera is:

$$e = u_observed - \pi(K, R_i, t_i, P)$$

where u_observed is the pixel coordinate actually observed and π(·) is the projection function of Chapter 9 (projecting the three-dimensional point P into the image with camera pose R_i, t_i and intrinsics K). BA minimizes the sum of squares of all reprojection errors over all camera poses and all three-dimensional points at once.

4. The Iterative Solving Procedure (Gauss-Newton / LM)

Figure 12-5 The Iterative Solving Procedure of Graph Optimization

① Initialization: string together a rough set of poses from odometry (the initial guess X_0)

↓

② Linearization: compute all residuals and Jacobians near the current X

↓

③ Build the linear system: $H \cdot \Delta x = -b$ (H being the sparse information matrix)

↓

④ Sparse solve: exploit sparsity to obtain the correction Δx quickly

↓

⑤ Update: $X \leftarrow X \oplus \Delta x$ (updating in pose space)

↓

⑥ Converged? No → return to ② / Yes → output the optimal solution X*

This whole procedure is what g2o, Ceres, and GTSAM carry out for us behind the scenes. Understanding the procedure is enough; there is no need to implement it by hand—just as driving does not require building the engine.

12.8 Contemporary Developments in Graph Optimization: Faster, Larger, and Less Afraid of Error

The framework introduced in this chapter has had essentially the same skeleton since it took shape around 2010. But it has kept advancing in three directions—speed, scale, and tolerance of erroneous data.

1. Incremental Solving: No Need to Recompute the Whole Graph Every Time

The most immediate question is this: a node is added with every step the robot takes, so must the whole graph be re-solved each time? GTSAM, mentioned in Section 12.6, answers with iSAM2—it organizes the factor graph into a Bayes tree, so that when a new edge is added only the small affected set of branches must be recomputed while the rest stays as it was.

This turns optimization from batch processing into real-time incremental updating, and it is the key to running graph optimization on a robot. It also explains a familiar phenomenon: the CPU is relaxed during ordinary mapping and suddenly becomes busy the moment a loop closure is detected—because a loop closure edge disturbs the whole tree.

2. Global Methods Return to the Stage

On the offline reconstruction side, a notable reversal has occurred recently. For a long time incremental reconstruction (adding images one at a time with repeated BA) was considered the most accurate but extremely slow, while global methods were fast but insufficiently stable. A 2024 ECCV work redesigned the global pipeline—view graph calibration, global positioning, and structure refinement in three steps, with camera positions and image points estimated together in the global positioning stage.

The result is striking: on a large dataset of thirty-six thousand images with cameras spanning over two hundred fifty meters, it completed reconstruction in 5.5 hours with ninety percent recall within one meter, whereas the traditional incremental method ran for more than seven days for only about fifty percent recall. Accuracy was comparable or better, at speeds several orders of magnitude greater.

The significance for students: if a project needs to reconstruct a campus or a building from thousands of photographs (a frequent need in the projects of Chapters 17 and 31), practical tools now exist and one need no longer wait days.

3. Robust Kernels: When the Front End Supplies a Wrong Loop Closure

Graph optimization has a fatal weakness, already mentioned under "the uniformity of constraints" in Section 12.3: it assumes every edge is correct. But the front end makes mistakes—the visual bag of words of Chapter 20 may mistake two similar-looking corridors for the same place, and the point cloud registration of Chapter 17 may converge to a wrong solution in a symmetric structure.

How serious is one wrong loop closure edge? It acts like a powerful rubber band, yanking two stretches of trajectory that are not in fact adjacent together and distorting the whole map on the spot. And because least squares penalizes large residuals quadratically, that erroneous edge has the loudest voice of all—an inherent fragility of the least-squares method.

Countermeasures come at two levels. The first is a robust kernel: replace the squared penalty on the residual with a more slowly growing function such as Huber or Cauchy, so that the influence of large residuals is suppressed and outliers no longer dominate the solution. The second is more thorough—let the optimization itself decide whether to believe an edge, for instance by attaching a switch variable to each loop closure edge that turns off automatically if it conflicts severely with the other constraints.

This matters a great deal in practice. If, while mapping with real hardware in Chapter 29, you find that "the map was fine and then suddenly distorted entirely," the first thing to suspect is a misjudged loop closure. And turning on the solver's robust kernel will

often resolve it.

4. Combination with Learning-Based Methods

Finally, it is worth remarking that the division of labor between graph optimization and deep learning is settling into shape. The most mature pattern today is not to replace optimization with a network but to let the network take the front end—detecting and matching features, judging loop closures, estimating depth priors—and hand the results to a traditional factor graph back end to solve.

The reasoning for this division is exactly that of Sections 7.8, 9.8, and 11.8: the geometric and probabilistic structure of the back end supplies interpretability, uncertainty, and mathematical guarantees, while the perceptual part of the front end goes to models that excel at learning regularities from data. ORB-SLAM3 in Chapter 23 and AI-assisted navigation in Chapter 32 can both be understood within this framework.

A Practical Order of Debugging

The map is distorted → suspect a wrong loop closure edge first (turn on the robust kernel, or raise the verification threshold for loop closures).

The map is skewed overall but locally correct → suspect a systematic bias in odometry (as in the left panel of Figure 12-2); the wheel diameter or wheelbase needs calibrating (Chapter 29).

No loop closure is detected → the problem is in the front end, not the back end (the branch and bound of Chapter 15, the visual bag of words of Chapter 20).

Solving is slow → check whether a dense solver has been used by mistake, or whether the node count calls for keyframe selection.

Case Studies

Case 12-1 The Power of Loop Closure: How a Crooked Map Is Straightened

This is the most dramatic demonstration of graph optimization, and nearly every SLAM classroom shows it. A robot goes once around a building, mapping with odometry and scan matching alone. Halfway round, the map still looks reasonable; but as the circuit nears completion, accumulated drift makes the map skew—a corridor that should close has its end offset from its start by several meters, and the same wall is even drawn as two diverging lines.

Then loop closure detection fires: the system recognizes "what I see now is the lobby I started from" and produces a loop closure edge. Graph optimization runs at once—and on screen the students watch the whole crooked map snap straight: the diverging walls become one, the offset corridor joins up, and the whole trajectory turns regular. In that instant the abstract description of Section 12.2, "spreading the contradiction over the whole trajectory," becomes a living picture before their eyes.

The impact of this case is that it lets students see with their own eyes how one edge corrects an entire map. This a filter cannot do—a filter has not retained its historical poses and cannot go back. The whole value of graph optimization is laid bare in that single moment of straightening. When implementing Cartographer in Chapter 15, students will trigger that moment with their own hands.

Case 12-2 The Lineage of GTSAM: From Classroom to Global Industry

As noted when introducing the CMU 16-833 course in Chapter 2: the GTSAM library developed by Professors Michael Kaess and Frank Dellaert packaged the theory

of factor graphs and incremental smoothing into an open-source tool, and it is now standard equipment in robotics companies and laboratories worldwide. The content of this chapter is the key to understanding what GTSAM does.

One key innovation of GTSAM is iSAM2 (incremental smoothing and mapping). Traditional graph optimization must re-solve the whole problem whenever new data arrives; iSAM2 cleverly updates only the part affected by the new data, which raises efficiency greatly and lets large-scale, long-duration SLAM run in real time. This addresses the pain point of "graph optimization is computationally heavy" raised in Section 12.7.

This case ties several threads of the book together: the leading university courses of Chapter 2, the graph optimization mathematics of this chapter, and real-world industrial application. It tells students that the mathematics they are studying is not an ivory-tower toy but the actual engine driving underwater robots, self-driving cars, and AR devices. Understanding this chapter puts you on the same footing of knowledge as those leading engineers.

Case 12-3 From Surveying to SLAM: The Century-Old Lineage of Bundle Adjustment

Bundle adjustment is often mistaken for a recent invention of computer vision, but its lineage traces back to nineteenth-century photogrammetry. The surveyors of that era reconstructed terrain from several aerial photographs and faced exactly the same problem: how to adjust camera positions and ground points so that the projections in all the photographs agree. The adjustment theory they developed is the direct ancestor of today's BA.

This history offers two lessons. First, the ideas of least squares and adjustment endure—from nineteenth-century surveying through twentieth-century Apollo navigation (Chapters 10 and 11) to twenty-first-century SLAM, humanity's core method for handling several erroneous measurements descends in one line. Second, knowledge across fields often converges: surveying, photogrammetry, computer vision, and robotics all meet at the point of BA. This echoes the observation in Chapter 10 that robot arms and mobile robots share one set of mathematics—mathematics is the common language of every engineering field, and mastering the fundamentals opens doors everywhere.

Hands-On Activities

Exercise 12-1 Optimizing a Pose Graph by Hand

Purpose: through reasoning on paper and implementation in code, to understand first-hand how a pose graph is straightened and to feel the power of a loop closure constraint. This exercise has a paper part and a coding part and suits teaching at various depths.

Part One: A Pose Graph on Paper (everyone)

1. On graph paper, draw a square patrol: starting from the origin, odometry says "4 squares right, 4 up, 4 left, 4 down" and the robot should return to the origin. First draw the four corner poses according to odometry.
2. Add drift: suppose each corner turns a little too little (85 degrees rather than 90, say), redraw the trajectory, and observe how the endpoint fails to return to the origin (a drift gap appears).

3. Add the loop closure edge: declare that the endpoint is the start. Reason intuitively: to bring the endpoint back to the start while disturbing the lengths of the four legs as little as possible, how should the four corners be adjusted? Draw the straightened trajectory by hand.
4. Discussion: the loop closure edge spreads the error over the four corners rather than concentrating the correction at one. Why is spreading better than concentrating?

Part Two: Pose Graph Optimization in Python (undergraduate)

5. Build a simple 2D pose graph in Python: several pose nodes, odometry edges connecting them (with simulated drift), and one loop closure edge.
6. Implement the residual computation: for each edge, compute the difference between the measured relative pose and the currently estimated relative pose.
7. Solve with an off-the-shelf optimizer (scipy.optimize.least_squares, or the Python interface of g2o or GTSAM) to minimize the sum of squared residuals.
8. Plot the trajectories before and after optimization together and observe how the drifted trajectory is pulled back into closure by the loop closure edge.
9. Experiment with the information matrix: set the weight (information matrix) of the loop closure edge large and then small, and observe how the strength of the straightening changes. With a large weight the system believes the loop closure more and pulls harder.

Part Three: Observing Loop Closure in Real SLAM (for those with a ROS environment)

10. Using Cartographer (Chapter 15) or another SLAM system supporting loop closure, drive the robot once around a simulated environment and watch in

RViz in real time: the map drifts gradually during the circuit, and the instant loop closure fires the map is straightened.

11. Record that moment of straightening, compare it with Case 12-1 and Section 12.2, and identify which edges are odometry edges and which is the loop closure edge.

Discussion Questions

1. In Part One, how did you adjust the four corners after adding the loop closure edge? Compared with charging all the error to the final leg, which map is more reasonable when the optimization spreads the error, and why?
2. In Part Two, how did the strength of the straightening change when you raised the weight of the loop closure edge? If the loop closure detection were in fact a misjudgment (a false loop closure), what would a high weight cause?
3. This chapter says graph optimization can go back and correct history while the filters of Chapter 11 cannot. From your experimental results, explain how that difference affects the quality of the final map.
4. (Undergraduate) Why does sparsity let a large pose graph be solved quickly? What would happen if every edge involved all the nodes (no sparsity)?

Extended Challenge (Undergraduate)

Install the Python bindings of g2o or GTSAM, load a public standard pose graph dataset (the intel or manhattan datasets in g2o format, for example), run the optimization, and visualize the difference before and after with matplotlib. Then try adding a wrong loop closure edge and observe how it wrecks the whole map—this will help you understand why Chapters 20 and 22 take such pains over robust loop closure detection and why robust kernels matter.

Further Reading

1. G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010. (The most classic introduction to graph-based SLAM and the principal basis of this chapter.)
2. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Lecture 6 (nonlinear optimization) and Lectures 9–10 (the back end, BA, and pose graphs). (The clearest material on implementing graph optimization in the Chinese-speaking world.)
3. R. Kümmerle et al., "g2o: A General Framework for Graph Optimization," ICRA, 2011. (The original paper on the g2o library.)
4. F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," Foundations and Trends in Robotics, 2017. (The authoritative work on GTSAM and factor graphs, freely available.)
5. B. Triggs et al., "Bundle Adjustment — A Modern Synthesis," Vision Algorithms, 2000. (The authoritative survey of bundle adjustment.)
6. The official Ceres Solver documentation (ceres-solver.org) and the official GTSAM tutorials (gtsam.org). (First-hand implementation guidance for the three major libraries.)
7. The factor graph and graph optimization units of the CMU 16-833 Robot Localization and Mapping course (introduced in Chapter 2). (Graduate-level material on graph optimization.)

The Close of Part III: From Mathematics Back to the Robot

The three chapters of Part III end here. Chapter 10 gave the language for expressing

pose, Chapter 11 the language for describing uncertainty, and this chapter the method of solving. Together they are the mathematical skeleton of all modern SLAM.

Looking back, these three chapters in fact answer three aspects of one question. "Where am I?"—Chapter 10 says it is a pose T; Chapter 11 says it is really a distribution; this chapter says that to find it, put all the constraints together and solve a least-squares problem.

From the next chapter, Part IV puts these tools genuinely to work. Chapter 13 turns LiDAR scan data into an occupancy grid map—using the coordinate transformations of Chapter 10, the probabilistic updates of Chapter 11, and this chapter's idea of finding a consistent solution from many observations all at once. Gmapping in Chapter 14 is an implementation of the particle filter, and Cartographer in Chapter 15 an implementation of this chapter's pose graph.

In other words, what Part III teaches is not "mathematics" but the tools every chapter of the next six parts will use. If a concept still feels abstract now, do not worry—when you watch a crooked map snap straight at the moment of loop closure in Chapter 29, everything in this chapter will become concrete.

Summary

1. Modern SLAM adopts a front-end-plus-back-end architecture: the front end turns sensor data into geometric constraints (scan matching, feature matching, loop closure detection) and the back end finds the most consistent solution under all the (often contradictory) constraints; graph optimization belongs to the back end.

2. A pose graph describes SLAM with nodes (the poses and landmarks to be estimated) and edges (relative measurement constraints); odometry edges string poses together like a chain, while loop closure edges form loops and are the key to eliminating accumulated drift.
3. Each edge has a residual (measurement minus prediction) and an information matrix (a trustworthiness weight, the inverse of the covariance); graph optimization unifies sensor constraints of every kind into weighted edges, realizing tight coupling of multiple sensors.
4. SLAM is cast as a least-squares problem: find a set of states minimizing the weighted sum of squared residuals; the square has both a mathematical justification (differentiability) and a probabilistic one (equivalence to maximum a posteriori under Gaussian assumptions), it is solved iteratively by Gauss-Newton or LM, and sparsity makes large scenes tractable.
5. Bundle adjustment optimizes camera poses and three-dimensional points together by minimizing reprojection error and is the source of accuracy in visual SLAM; modern systems combine local BA (high accuracy) with a global pose graph (fast loop closure).
6. The three major libraries do the solving for us: g2o (general graph optimization, used by ORB-SLAM2), Ceres (Google's general least-squares library, used by Cartographer), and GTSAM (factor graphs plus incremental iSAM2); the factor graph is a generalization of the pose graph.
7. Modern SLAM mostly adopts graph optimization because it can go back and correct history, raise accuracy by repeated relinearization, exploit sparsity to handle large scenes, and fuse multiple sensors uniformly; the price is greater

memory and computation, and it became mainstream only after hardware and algorithms had advanced.

Part III, "The Mathematical Foundations of SLAM," is complete. We have assembled the full mathematical toolbox: coordinate transformation (Chapter 10), probability and filtering (Chapter 11), and optimization and graph optimization (this chapter). These three chapters are abstract yet are the bedrock of everything. From the next part onward the theory lands on real robots—Part IV begins with the most approachable subject, 2D SLAM and the cleaning robot, and you will see how Gmapping applies the particle filter of Chapter 11 and how Cartographer applies the graph optimization of this chapter. Mathematics and practice are about to meet.